

OHJR: Escape Room Game

Graduation Project (1)

by

Hind	443
Raghad	443
Reema	443
Bayan	443
Joud	443
Reenad Mohamed Saeed Alqahtani	443
Rana	443

**A project submitted in partial fulfilment of the requirements for
the degree of Bachelor of Science (Computer Science)**

Supervised by

MS. Dalia Assiri

1st Trimester - Academic Year 1446 (2024)

Abstract

In our project we will study how to design and implement an escape room game. We will create a website that educates users about cybersecurity where players can enter the game and immerse themselves in it by answering a specific number of questions within a specific period of time, focusing on its entertainment value and educational importance. And we can use these algorithms that will help us facilitate the process of calculating a player's scoring with Time Factor algorithms and basic Scoring Algorithm (Points-Based Games).

Keywords escape room game; website; game; players; enjoyment

Table of Content

Abstract	2
List of figures	8
List of Tables.....	10
List of Abbreviations.....	11
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Problem statement	1
1.3 Motivation	1
1.4 Proposed system	1
1.5 Aim of the project.....	2
We noticed that with the increasing use of the digital world, security problems and threats increased due to a lack of awareness. For this reason, we decided to create a website to help users increase awareness in the field of cybersecurity.	2
1.6 Project Objectives	2
• Designing a set of puzzles and challenges for different levels of difficulty to educate people about cyber security and providing tools that can help them solve them.	2
• Developing the website and making it attractive, educational and fun at the same time.....	2
1.7 Project Scope.....	2
• Our website aims to design a game that attracts players, especially university students, because they are the future generation, as they must solve puzzles in a limited period of time.	2
1.8 Significance of the project.....	2
1.9 Software development methodology	2

1.10	project Organization	4
1.10.1	Chapter Organization	4
1.10.2	Project Timeline.....	4
1.10	Summary	6
Chapter 2:	Background and literature review	7
2.1	History	7
2.2	literature review.....	7
2.2.1	Education pleasant one game	7
2.2.2	Der Einfluss didaktischer game	9
2.2.3	Project Phoenix game.....	9
2.3	Comparison table between existing system and proposedsystem	11
2.4	Feasibility analysis	12
2.4.1	Technical feasibility	12
2.4.2	Economic Feasibility.....	12
2.5	Requirements Analysis	13
2.5.1	Functional Requirements	13
2.5.2	NON- Functional Requirements	14
2.6	Summary	15
Chapter 3:	System Analysis and SystemDesign	16
3.1	Introduction.....	16
3.2	Interaction Analysis	16
3.2.1	Uml Models And Diagrams	16
3.3	Design Concept.....	25
3.4	Data Flow Diagrams for Level 0	25
3.5	Data Flow Diagrams for Level 1	26

3.6 Database Design.....	26
3.6.1 ER Diagram	27
27	
3.6.2 Database Tables	28
3.7 Sample interface design.....	31
3.8 Underlying Techniques	38
3.9 Summary	41
Chapter 4: System Implementation	42
4.1 Introduction.....	42
4.2 Software Tools Used in Implementation.....	42
4.3 Database Implementation.....	42
4.4 Database Design.....	43
4.4.1 Users Table	43
4.4.2 Score Game Table.....	44
4.4.3 Questions Table	45
4.4.4 Levels Table.....	45
4.4.5 Games Table	45
4.4.6 Answers Table	46
4.5 Interface Design	46
4.5.1 Home Interface.....	46
4.5.2 Sign-up Interface.....	47
4.5.3 Login Interface.....	48
4.5.4 Levels Interface.....	48
4.5.5 Room 1 Interface.....	49
4.5.6 Room 2 Interface.....	49

4.5.7 Room 3 Interface.....	50
4.5.8 Add Level for Admin Interface.....	50
4.5.9 Admin Add Puzzle Interface.....	51
4.5.10 Admin Levels List Interface	51
4.5.11 Admin Puzzles List Interface	52
4.6 Sample Code	53
4.6.1 Home Code	53
4.6.2 Sign-up Code	53
4.6.3 Login Code.....	54
4.6.4 Levels Code	55
4.6.5 Room 1 Code	56
4.6.6 Room 2 Code	57
4.6.7 Room 3 Code	58
4.6.8 Add Level for Admin Code	59
4.7.9 Admin Add Puzzle Code	59
Chapter 5: System Testing	61
5.1 Introduction.....	61
5.2 Functional Testing	61
5.2.1 Unit Testing	61
5.2.2 Integration Testing.....	61
5.2.3 Regression Testing	61
5.3 System Testing.....	62
5.3.1 Test Cases	62
Chapter 6: Conclusion and Future Works.....	70
6.1 Conclusion	70

6.1 Future Works	70
References	71

List of figures

Figure 1. 1: Agile development methodology	3
Figure 2. 1: the puzzle design	8
Figure 2. 2: the puzzle composition	7
Figure 2. 3: Puzzle organization in Project Phoenix.....	10
Figure 3. 1: Use Case Diagram	17
Figure 3. 2: Class Diagram	18
Figure 3. 3: Sequence Diagram for Sign-Up	19
Figure 3. 4: Sequence Diagram for Login	20
Figure 3. 5: Sequence Diagram for Playing a Puzzle Game.....	21
Figure 3. 6: Sequence Diagram for Adding a Puzzle Game	22
Figure 3. 7: Activity Diagram for Player	23
Figure 3. 8: Activity Diagram for Admin	24
Figure 3. 9: Context Flow Diagrams.....	25
Figure 3. 10: Data Flow Diagram Level 1	24
Figure 3. 11: ER DIAGRAM.....	25
Figure 3. 12: logo of our game.....	30
Figure 3. 13: log in page	32
Figure 3. 14: sign in page.....	32
Figure 3. 15: home page in our website.....	33
Figure 3. 16: home page.....	33
Figure 3. 17: Example of room	34
Figure 3. 18: design the model of escape room	33
Figure 3. 19: start game	35
Figure 3. 20: Example 1 of question.....	35
Figure 3. 21: Example 2 of question.....	36
Figure 3. 22: about the escape room	35
Figure 3. 23: about easy level question.....	36
Figure 3. 24: about medium level question.....	36
Figure 3. 25: about hard level question.....	36

Figure 4. 1: Users Table.....	44
Figure 4. 2: Score Game Table	44
Figure 4. 3: Questions Table.....	45
Figure 4. 4: Levels Table	45
Figure 4. 5: Games Table.....	46
Figure 4. 6: Answers Table.....	46
Figure 4. 7: Home Interface	47
Figure 4. 8: Sign-up Interface	47
Figure 4. 9: Login Interface	48
Figure 4. 10: Levels Interface	48
Figure 4. 11: Room 1 Interface	49
Figure 4. 12: Room 2 Interface	49
Figure 4. 13: Room 3 Interface	50
Figure 4. 14: Add Level for Admin Interface	50
Figure 4. 15: Admin Add Puzzle Interface	51
Figure 4. 16: Admin Levels List Interface	52
Figure 4. 17: Admin Puzzles List Interface	52
Figure 4. 18: Home Code.....	53
Figure 4. 19: Sign-up Code.....	54
Figure 4. 20: Login Code	54
Figure 4. 21: Levels Code.....	55
Figure 4. 22: Room 1 Code.....	56
Figure 4. 23: Room 2 Code.....	57
Figure 4. 24: Room 3 Code.....	58
Figure 4. 25: Add Level for Admin Code.....	59
Figure 4. 26: Admin Add Puzzle Code.....	60

List of Tables

Table 1. 1: Project Timeline.....	4
Table 2. 1: Comparison between existing system and proposed system	10
Table 3. 1: Table 1: Game Developer	28
Table 3. 2: Table 2: Player	28
Table 3. 3: Table 3: Puzzle Game	29
Table 3. 4: Table 4: Puzzle Level	29
Table 3. 5: Table 5: Question.....	29
Table 3. 6: Table 6: Answer.....	30
Table 3. 7: Table 7: Score Game	30
Table 5. 1: Test case: an invalid sign-up attempt, where the username field is left empty	62
Table 5. 2: Test case: an invalid sign-up attempt, where the password field is left empty	63
Table 5. 3: Test Case: an invalid login, where the email field is left empty	64
Table 5. 4: Test Case: an invalid login, where the password field is left empty	65
Table 5. 5: Test Case: email error, where the email is not correct	66
Table 5. 6: Test Case: password error, where the password is not correct	67
Table 5. 7: Test Case: an invalid add puzzle	68
Table 5. 8: Test Case: an invalid add level	69

List of Abbreviations

ERG	Escape Room Game
HTM L	Hyper Text Markup Language
SQL	Structured Query Language
DFD	Data flow diagram
ERD	Entity relation diagram
CSS	Cascading Style Sheet

Chapter 1: Introduction

1.1 Introduction

Over the past 10 years, escape rooms have become a global phenomenon — transitioning from literal pen and paper puzzles to engaging narrative-driven experiences that appeal to anyone. Escape rooms are, at their core, time-fixed challenges that require players to complete a specific set of tasks or solve puzzles in order to "escape" from a themed environment. [1]

The Escape Room is a physical and social adventure game where players are set into a specially designed space to use their detective skills to decipher clues, solve puzzles, and complete tasks until they have either opened the final door within the time limit or failed. Born out of video games, the idea has spawned a popular real-world recreation among friends, and family or corporate team-building.

1.2 Problem statement

In our time today, with the increased reliance on the Internet by individuals and official bodies, threats, electronic vulnerabilities, and cyber attacks have increased, such as the threat of data confidentiality, malicious viruses, and phishing, and this leads to large financial losses and violations of privacy, and all of this is due to the few Public awareness of the importance of cybersecurity and lack of sufficient knowledge of how to deal with private information and data , That's why create an educational game that raises awareness towards cybersecurity.

1.3 Motivation

Escape room games effectively address a range of social, cognitive, and emotional needs. By providing unique, engaging experiences, and problem-solving, they have become a popular choice for entertainment and personal development. As they continue to evolve, escape rooms will likely adapt to meet the changing preferences of participants, further solidifying their place in the entertainment landscape.

1.4 Proposed system

Our system is supposed to build a website that includes a game that helps university students focus their information through the questions that are covered in the game, which range from easy, medium and difficult levels.

1.5 Aim of the project

We noticed that with the increasing use of the digital world, security problems and threats increased due to a lack of awareness. For this reason, we decided to create a website to help users increase awareness in the field of cybersecurity.

1.6 Project Objectives

- Designing a set of puzzles and challenges for different levels of difficulty to educate people about cyber security and providing tools that can help them solve them.
- Developing the website and making it attractive, educational and fun at the same time.

1.7 Project Scope

- Our website aims to design a game that attracts players, especially university students, because they are the future generation, as they must solve puzzles in a limited period of time.

1.8 Significance of the project

- enjoying the game
- Improving skills and thinking about solving problems
- Giving a scientific description of the importance of the room escape game as an educational tool

1.9 Software development methodology

In our project, we will implement Agile development methodology to effectively accomplish our project objectives. Agile methodologies are recognized for their ability to enhance

transparency by rendering project progress visible to all team members. This transparency is particularly advantageous in teams comprising individuals with diverse backgrounds, expertise, and skills, as it fosters greater collaboration and knowledge sharing, ultimately leading to improved quality outcomes. Moreover, the Agile approach enables us to identify and mitigate potential risks at an early stage of the project, thereby facilitating proactive risk management and ensuring smoother project execution.

In our project, we've decided to use Agile methodology to help us reach our goals effectively. Agile methods are known for making project progress clear to everyone on the team, which is really helpful for us since we have team members with different backgrounds and skills. This transparency ensures better quality outcomes. Our project requires frequent testing and feedback to make sure we provide our service accurately and satisfactorily. So, we believe that going with Agile fits our project well and will help us succeed efficiently.

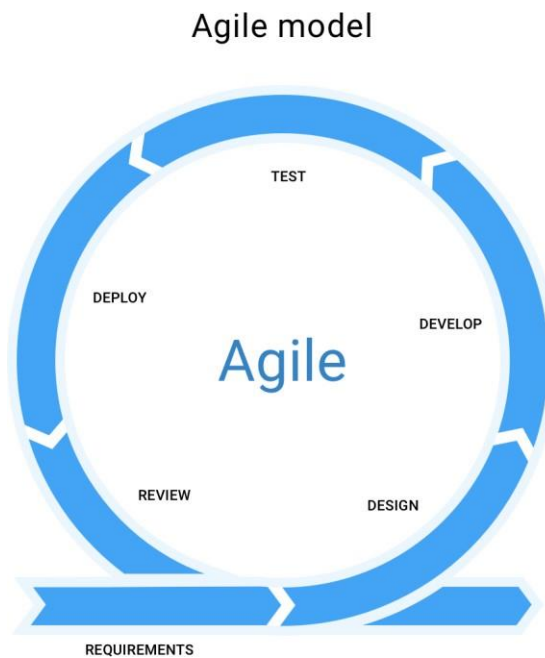


Figure 1. 1: Agile development methodology

1.10 project Organization

1.10.1 Chapter Organization

Chapter 2: Literature Review: Provides an extensive overview of existing literature pertaining to the project's subject matter.

Chapter 3: System Analysis: Focuses on evaluating current systems, identifying, and specifying both functional and non-functional requirements.

Chapter 4: System Design: Explores the architectural and object-oriented design of the system, encompassing data modeling and user interface design.

Chapter 5: The chapter focuses on the basic implementation of the project from the software and interfaces to testing the basic results of the escape room questions.

Chapter 6: Includes the summary or future vision of the project and recommendations directed to the players of the escape room game

1.10.2 Project Timeline

Table 1. 1: Project Timeline

Month	September			October				November		
Week	W 2	W 3	W 4	W 1	W 2	W 3	W 4	W 1	W 2	W 3
Group formation and title selection										
Determine the objectives and scope										
Study related works										
System Analysis										
System Design										

Conclusion and Future Work										
-------------------------------	--	--	--	--	--	--	--	--	--	--

1.10 Summary

In this chapter, we have set out the main points of the room escape project. We have discussed the importance and objectives of the project, in addition to the methodology used and the timetable for implementing and completing the project.

Chapter 2: Background and literature review

2.1 History

The escape room game first appeared in Japan in 2007, and later spread to European countries, where this game gained a lot of popularity due to the interactive entertainment between players and the time that controls the end of the game, which increases the challenge of solving questions and winning the game.

As science and knowledge advanced, the escape room game was further developed by integrating digital elements and artificial intelligence with websites, which increased the popularity and playability of this game.

2.2 literature review

The challenge continues due to the arduous nature of accessing information and its unavailability through old methods. As part of Saudi Arabia's Vision 2030, government bodies like Hadaq and Tamheer are helping students improve the education. They are using modern methods like personality tests, talking to professionals and providing information on their websites. It aims to make science and career simpler and more personalized for students. Therefore, our website aims to provide many tests within the scope of the College of Computer Science,

Many studies have studied the escape room game, perhaps the most important of which is:

2.2.1 Education pleasant one game

The author Markus Wiemker of this study relied on exploring the emotions of players through the challenges they encounter and their psychological impact on them. And investigating how to enhance cooperation and its impact on players, which enhances the transformation of negative emotions into positive emotions in the challenge to reach the goal. The results indicate that the immersive nature of escape rooms, along with cooperative challenges, enhances the sense of accomplishment and camaraderie among players. This research highlights the potential of escape rooms not only as entertainment but also as tools to enhance social interaction and emotional resilience in difficult situations [1].

In the case of physical challenges, this too should be completable based on items found within the room (If the players need to manipulate a physical artifact then the tools should be present in the room. For example, escape rooms cannot rely on players to bring their own screwdriver).

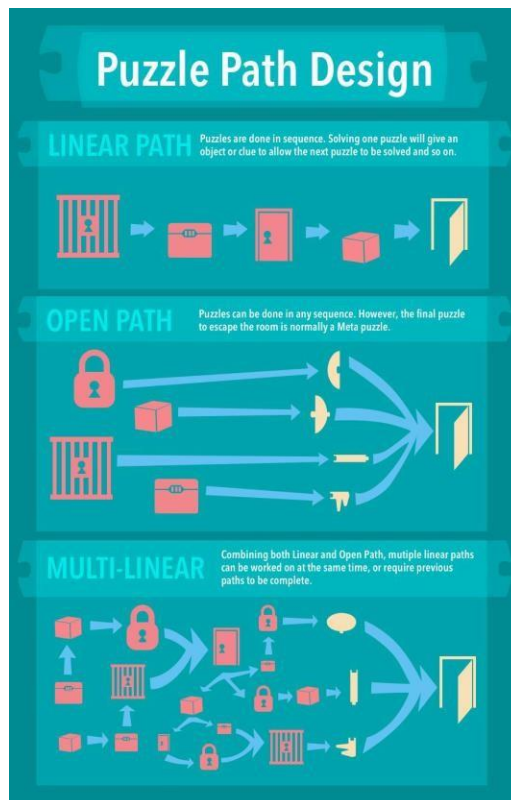


Figure 2. 1: the puzzle design

2.2.2 Der Einfluss didaktischer game

The authors Buchner, Rüter, & Kerres in This study combined the concepts of science and education with a focus on the effectiveness of escape room games and their impact on education. The authors used a mixed methods approach by integrating traditional science with game-enhanced learning, which facilitates access to information. The results revealed to the researchers that pre-educational participation in an escape room enhances motivation and curiosity, while post-educational participation enhances knowledge retention and application. This research highlights the importance of scientific innovation in increasing knowledge among students who are the main part of this game [2].

In this game, programmers using digital ERG were more effective when explicit instructions were provided before or after playing the game. In line with CLT.

2.2.3 Project Phoenix game

In this study, the authors Heikkinen and Shumeyko designed an escape room game based on attractive designs aimed at gaining knowledge with the complexity of the puzzles adopted in this game. The authors emphasize the importance of the design process in attracting players with the importance of sound and emotional effects that attract the player. This research contributes to increasing knowledge and provides an educational basis for teachers who seek to communicate information in an easy and simple way to students [3] .

The design of the puzzles for Project Phoenix was approached with create a set of unique and unexpected tasks that would not only fit the chosen theme and story, but become an essential part of them, helping propel the created narrative forward all the while making it more complex and meaningful. Variety and incorporation of multiple senses into the tasks were also paid great attention to in the design process.

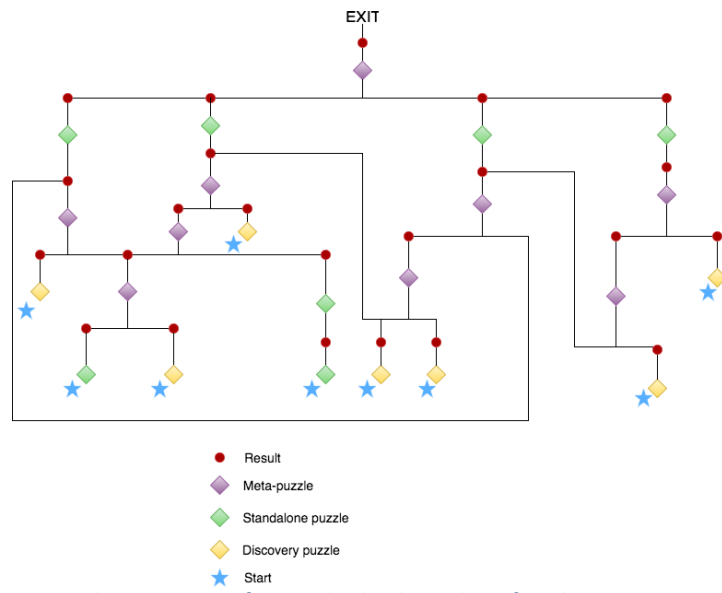


Figure 2. 2: Puzzle organization in Project Phoenix

2.3 Comparison table between existing system and proposed system

Table 2. 3: Comparison of the studied game systems with our proposed game system

Feature	Proposed System	Education pleasant one game	Der Einfluss didaktischer game	Project Phoenix game
Theme Variety	Our game depends on the questions of cyber security with the aspect of education and pleasure	The educator can create their own scenarios to be used with their class.	The used a mixed methods approach with the game.	The used cultural questions in the game.
Puzzle Complexity	different levels Easy, medium and difficult	Users said it was difficult to use	Users did not encounter any problems in using the site	Users did not encounter any problems in using the site
User Interfaces	Attractive and easy to use interfaces	Interfaces were not easy enough	Easy interface but unattractive	Attractive interfaces are designed with general questions
User Engagement	website	website	website	website
Educational Value	Yes	Yes	Yes	No
Target Group	University students	School students	University students	Adventurers

2.4 Feasibility analysis

2.4.1 Technical feasibility

2.4.1.1 Software Requirements

- Asana: For project management and puzzle organization.
- Canva: for designing intuitive user interfaces
- My SQL Server: for create and store data such as (player data, game progress)
- Visual studio code: for Web Development Frameworks:
 1. Front end for interfaces the escape room game. we will use (HTML, CSS, java script, React)
 2. Back end for real-time interactions is required we will use (PHP)

2.4.1.2 Hardware Requirements

- A smart device, laptop or computer in order for the player to be able to enter the game
- Input Devices like Keyboards and Mice for players to interact with the game
- High-quality speakers or sound systems
- Routers and switches to ensure stable internet connectivity for online escape room components.

2.4.2 Economic Feasibility

2.4.2.1 Software Requirements cost

We will rely on free software to design the escape room game, which serves our work, and this is what makes our project contain many challenges for us.

2.4.2.1 Hardware Requirements cost

The cost required for the devices varies depending on their type and quality, but we rely on the average quality of the devices that serve the game and are easy to use for the players.

2.5 Requirements Analysis

2.5.1 Functional Requirements

1. Functional Requirements for Player

- The player should be able to log in to the system.
- The player should be able to update their profile information.
- The player should be able to reset their password if forgotten.
- The player should be able to select a puzzle level (easy, medium, hard).
- The player should be able to start the puzzle.
- The player should be able to view the questions.
- The player should be able to solve the questions.
- The player should be able to view their score.
- The player should be able to end the game.

2. Functional Requirements for Game Developer (Admin)

- The game developer should be able to log in to the system.
- The game developer should be able to update their profile information.
- The game developer should be able to reset their password if forgotten.
- The game developer should be able to monitor time.
- The game developer should be able to manage puzzle levels by adding, updating, or deleting them.
- The game developer should be able to manage players by viewing, blocking, or unblocking player accounts.
- The game developer should be able to log out of the system.

2.5.2 NON- Functional Requirements

Performance Requirements

1. The system should load each page within **2 seconds** under normal network conditions.
2. The game should handle up to **500 concurrent players** without performance degradation.
3. Queries to fetch puzzles or scores should execute in **less than 1 second** on average.

Usability Requirements

1. The user interface should be intuitive and easy to navigate, with clear instructions for players and admins.
2. Error messages should be descriptive, providing actionable steps to resolve issues (e.g., invalid login credentials).
3. The game should provide responsive design, ensuring compatibility with both desktop and mobile devices.

Security Requirements

1. All sensitive data, including passwords, should be encrypted using **hashing algorithms like crypt.**

Scalability Requirements

1. The system should support adding more puzzle levels and players without requiring architectural changes.
2. The database design should allow for easy addition of new features, such as achievements or leaderboard functionalities.

Availability Requirements

1. The system should be operational **24/7**, with a maximum downtime of **4 hours per month** for maintenance.
2. Scheduled maintenance times should be communicated to users at least **24 hours in advance**.

Maintainability Requirements

1. The system should log errors and provide detailed reports for debugging, accessible only to admins or developers.
2. The application should use a **modular architecture** to allow easy

updates or additions to functionality.

Reliability Requirements

1. The system should have a **99.9% uptime**, with automated recovery mechanisms for server crashes.

2.6 Summary

In this chapter, we talked about previous research on the escape room game, discussed the requirements for the project, determined the cost, and finally talked about the functional and non-functional requirements of the system.

Chapter 3: System Analysis and System Design

3.1 Introduction

This chapter will utilize UML diagrams to depict the system's structure and outline key entities crucial for its operation, along with their interactions. We will also cover the database that will store the necessary data for the system's functionality. Furthermore, we will examine the fundamental system interfaces that facilitate user interaction. Finally, we will detail the core technologies planned for implementation in our proposed system.

3.2 Interaction Analysis

In UML, we can represent interactions between objects through various types of diagrams. Each diagram presents the same foundational information but offers a unique perspective on the details. Interaction diagrams specifically model how a group of objects collaborates to accomplish a specific task. They are particularly effective for analyzing the behavior of multiple objects within a single use case, illustrating the interactions among them clearly.

3.2.1 Uml Models And Diagrams

Real-world applications are represented through models and diagrams. Models offer an abstract representation of a system, whereas diagrams provide more concrete representations. In UML, models capture systems at various levels of detail; some models present the system at a higher, more abstract level, while others delve into greater detail.

In this section, we will present the Use Case Diagram, Class Diagram, Sequence Diagram, and Activity Diagram.

3.2.1.1 Use Case Diagram

Use case diagram shows the interactions between actors (users or external systems) and a system. It shows how different actors interact with the system to accomplish specific tasks or certain goals. The main components of the use case diagram are the actors, use cases, system boundaries, and relationships

The use case diagram of our system is shown in figure 3.1.

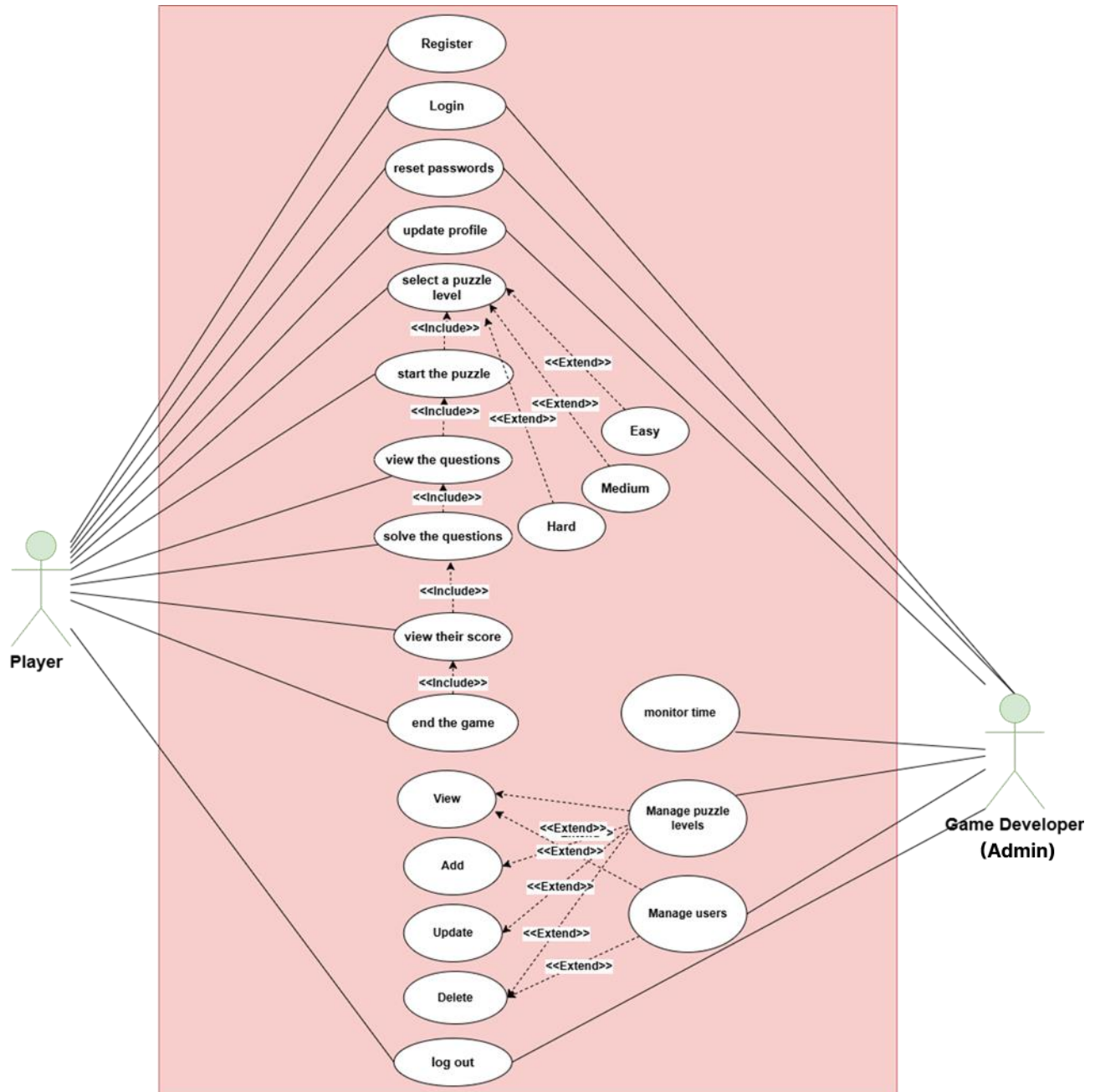


Figure 3. 1: Use Case Diagram

3.2.1.2 Class Diagram

A class diagram is a type of UML diagram that represents the static structure of a system by showing the classes in the system, their attributes, methods, and the relationships between them. It's used to visualize the system's structure and Design and is an essential tool for object-oriented software development. The following figure 3.2 shows the class diagram of the proposed system.

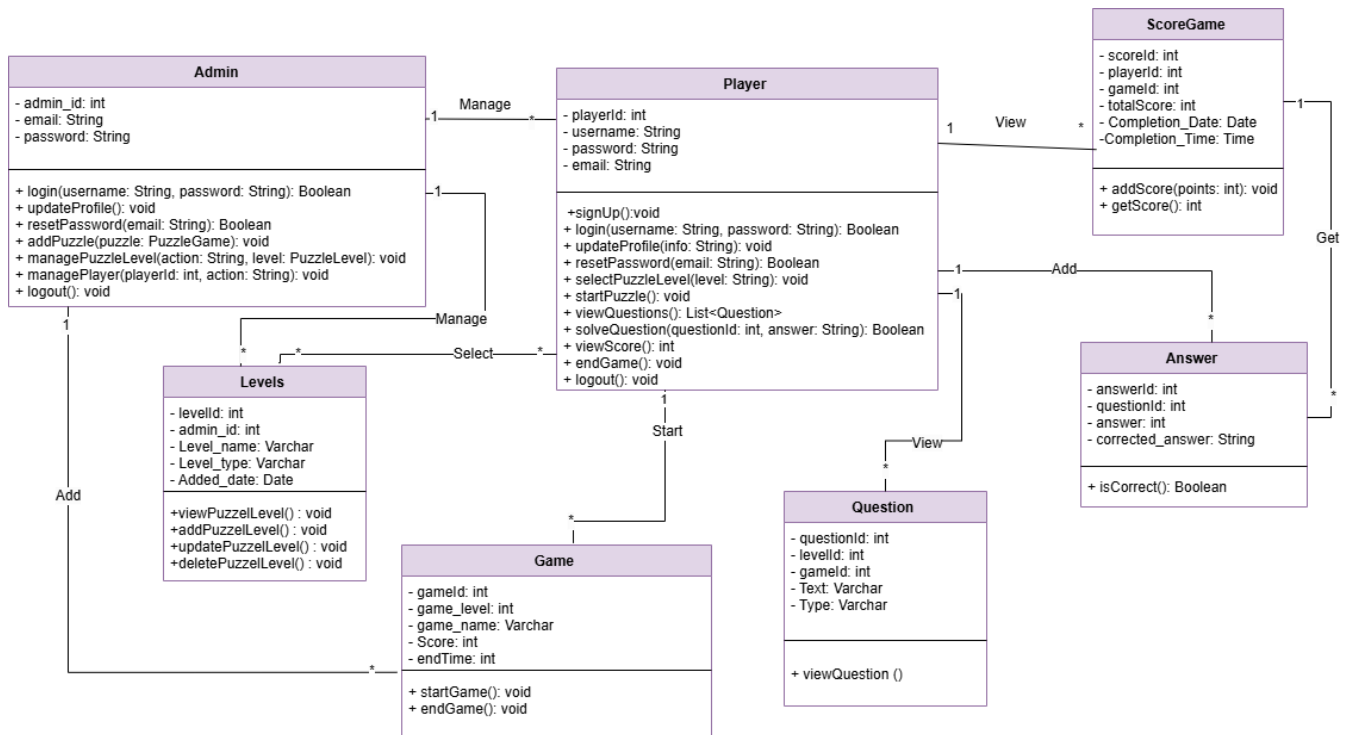


Figure 3. 2: Class Diagram

3.2.1.3. Sequence Diagram

This diagram shows the external actors, the messages or methods of these actors, the return values, and any loop pointers. The following diagram shows a sequence diagram of the proposed system.

3.2.1.3.1 Sequence Diagram for Sign-Up

Sequence diagram for sign-up is shown in Figure 3.3.

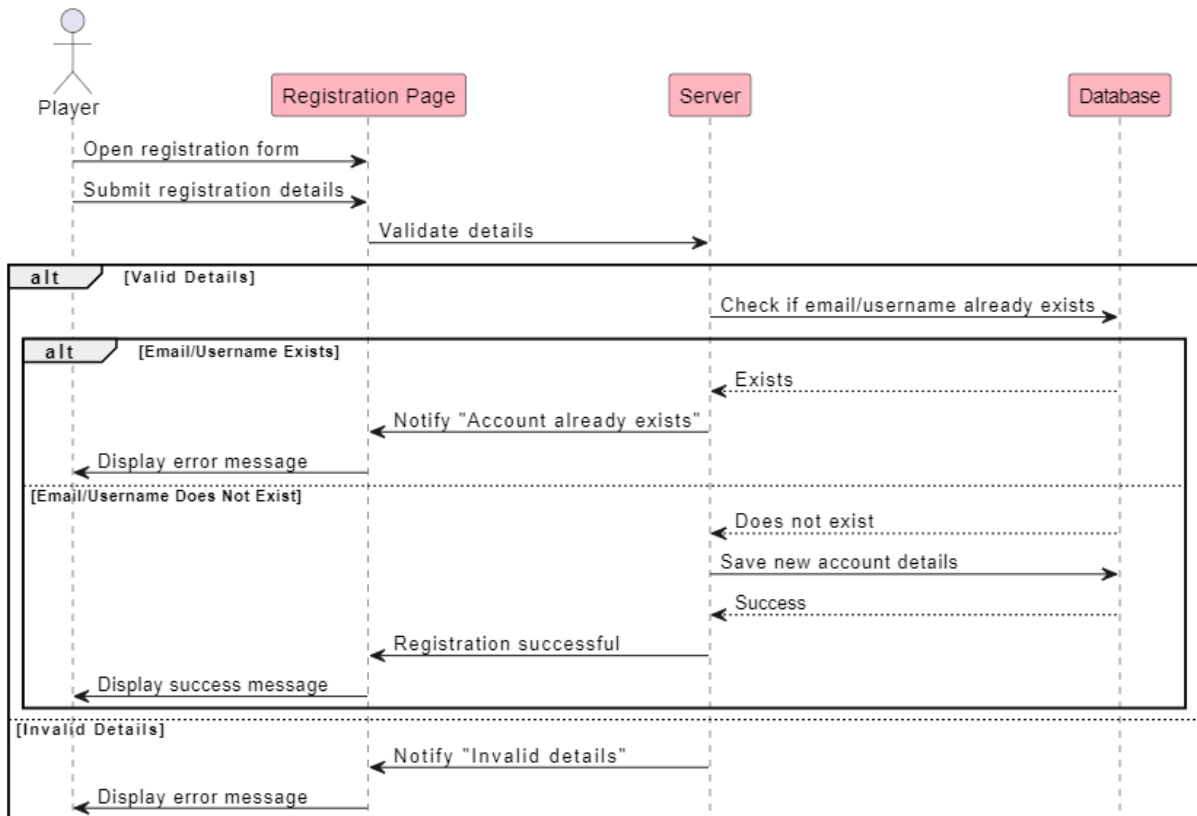


Figure 3. 3: Sequence Diagram for Sign-Up

3.2.1.3.2 Sequence Diagram for Login

Sequence diagram for login as shown in figure 3.4.

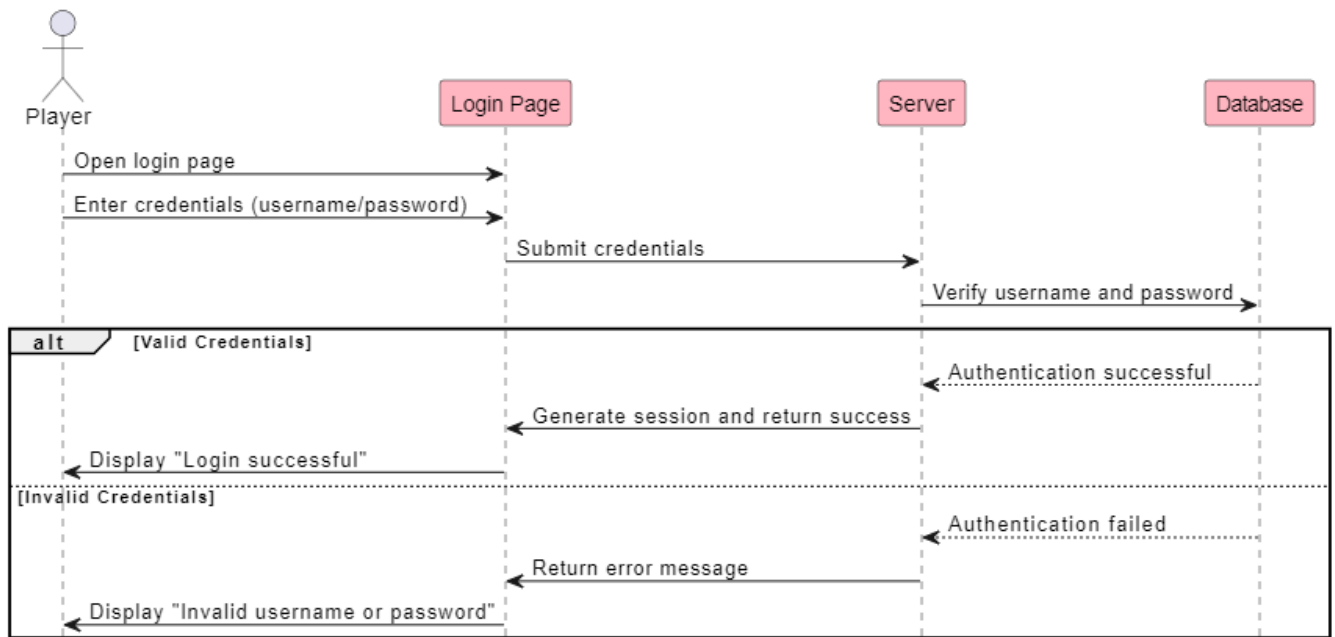


Figure 3. 4: Sequence Diagram for Login

3.2.1.3.3 Sequence Diagram for Playing a Puzzle Game

Sequence diagram for playing a puzzle game as shown in figure 3.5.

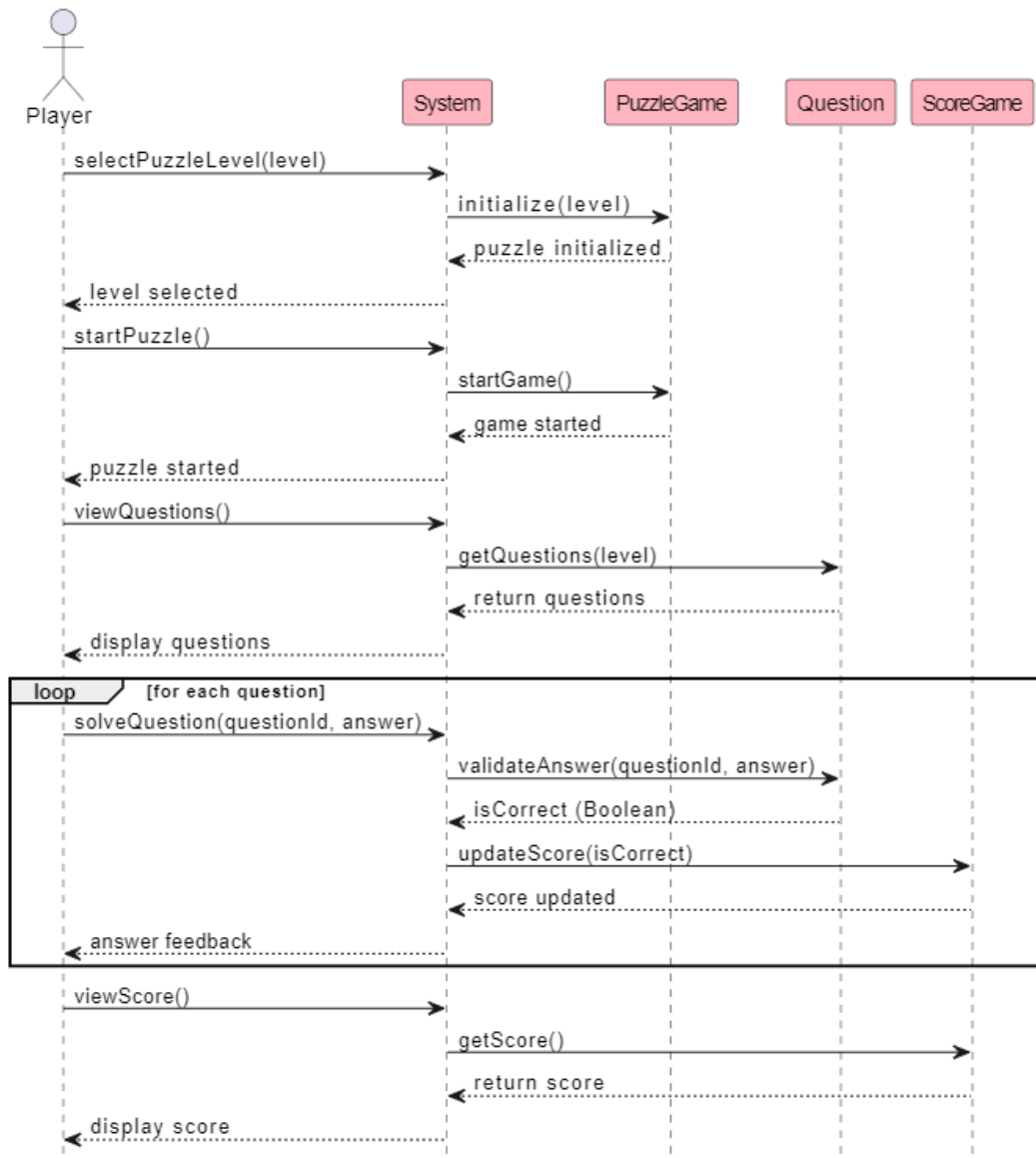


Figure 3. 5: Sequence Diagram for Playing a Puzzle Game

3.2.1.3.4 Sequence Diagram for Adding a Puzzle Game

Sequence diagram for adding a puzzle game as shown in figure 3.6

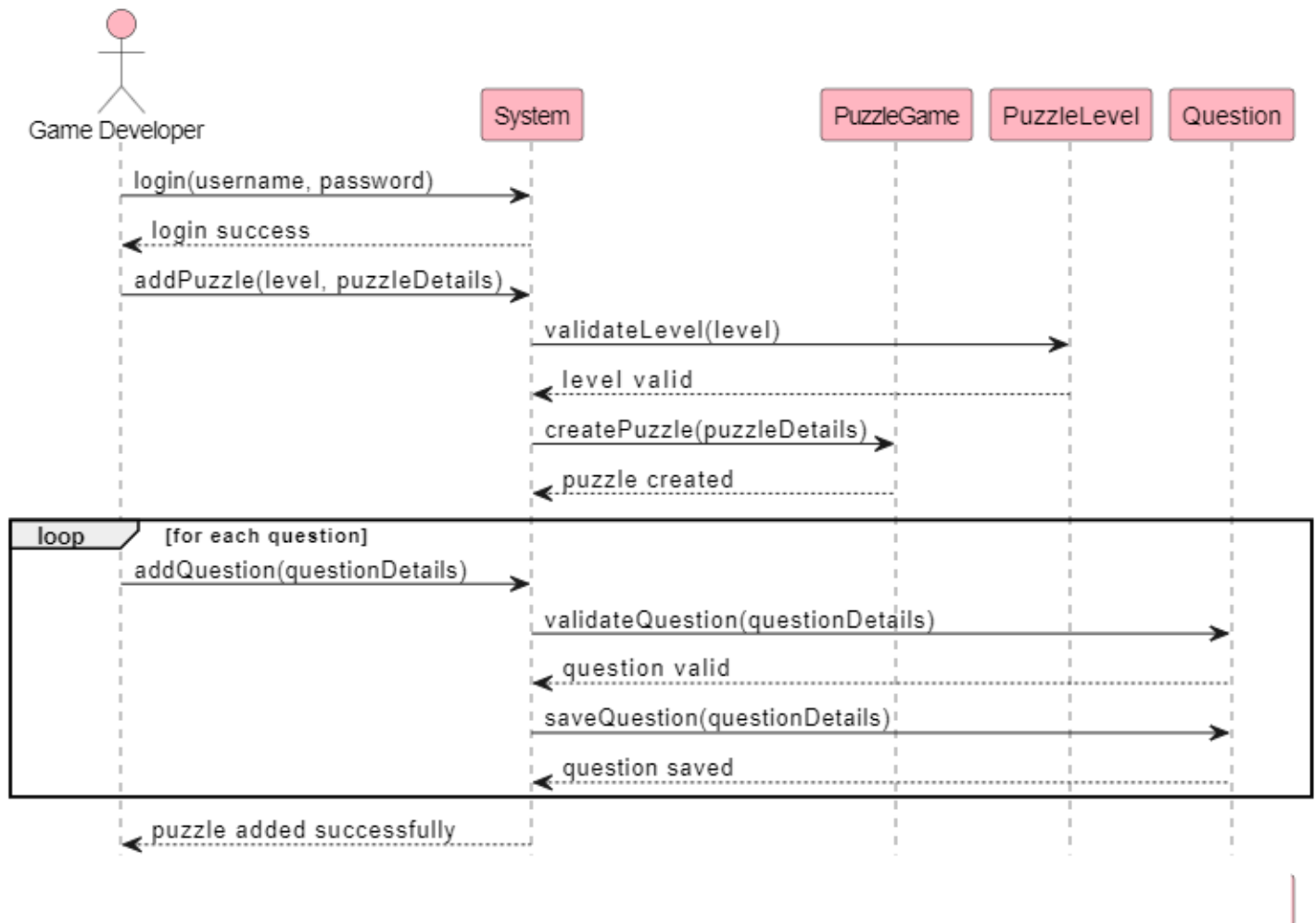


Figure 3. 6: Sequence Diagram for Adding a Puzzle Game

3.2.1.4 Activity Diagram

The activity diagram displays representations of workflows of the processes and actions. It is designed to visualize the data flows contributing to the relevant activities. The following sections depict the activity diagrams of the proposed system.

3.2.1.4.1 Activity Diagram for Player

Activity diagram for player as shown in figure 3.7



Figure 3. 7: Activity Diagram for player

3.2.1.4.2 Activity Diagram for Admin

Activity diagram for admin as shown in figure 3.8

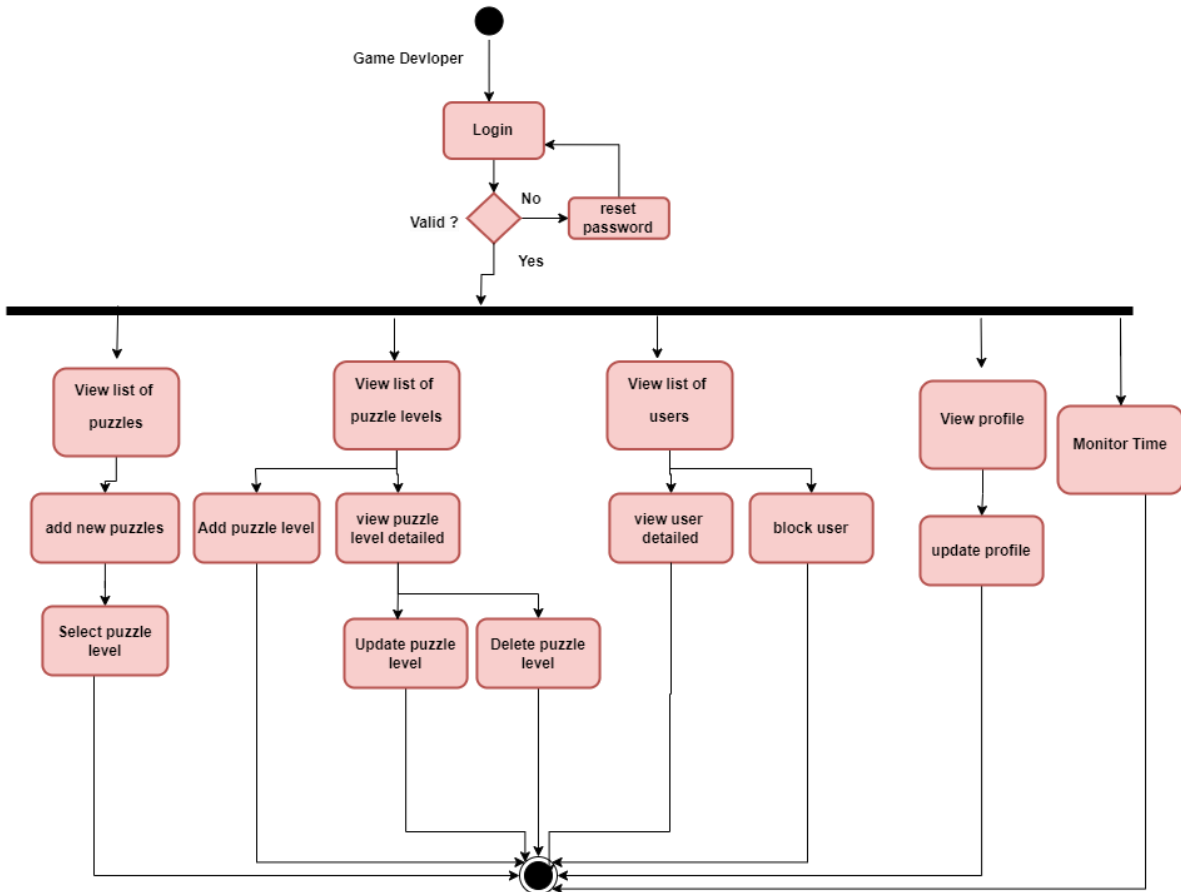


Figure 3. 8: Activity Diagram for Admin

3.3 Design Concept

System design is concerned with how to achieve the goal of the system to achieve the specific requirements of the user.

3.4 Data Flow Diagrams for Level 0

A data flow diagram (DFD) represents the flow of information through a process or system. Data flow diagrams represent systems and processes that are difficult to describe in words. Data flow diagrams are also classified according to their level as shown in figure 3.9.

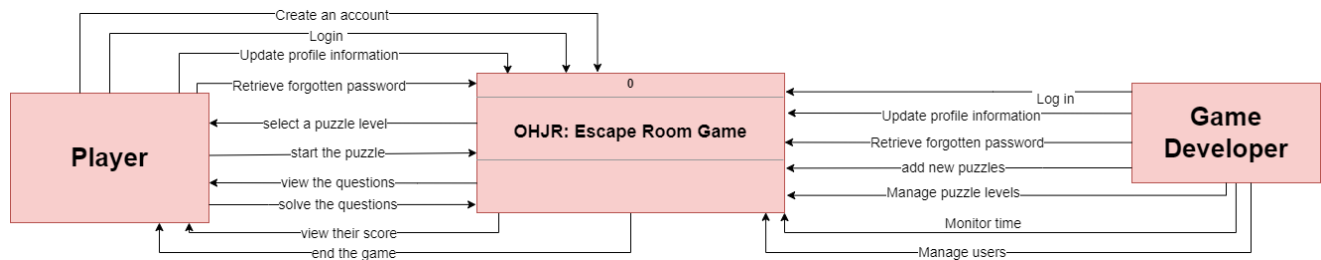
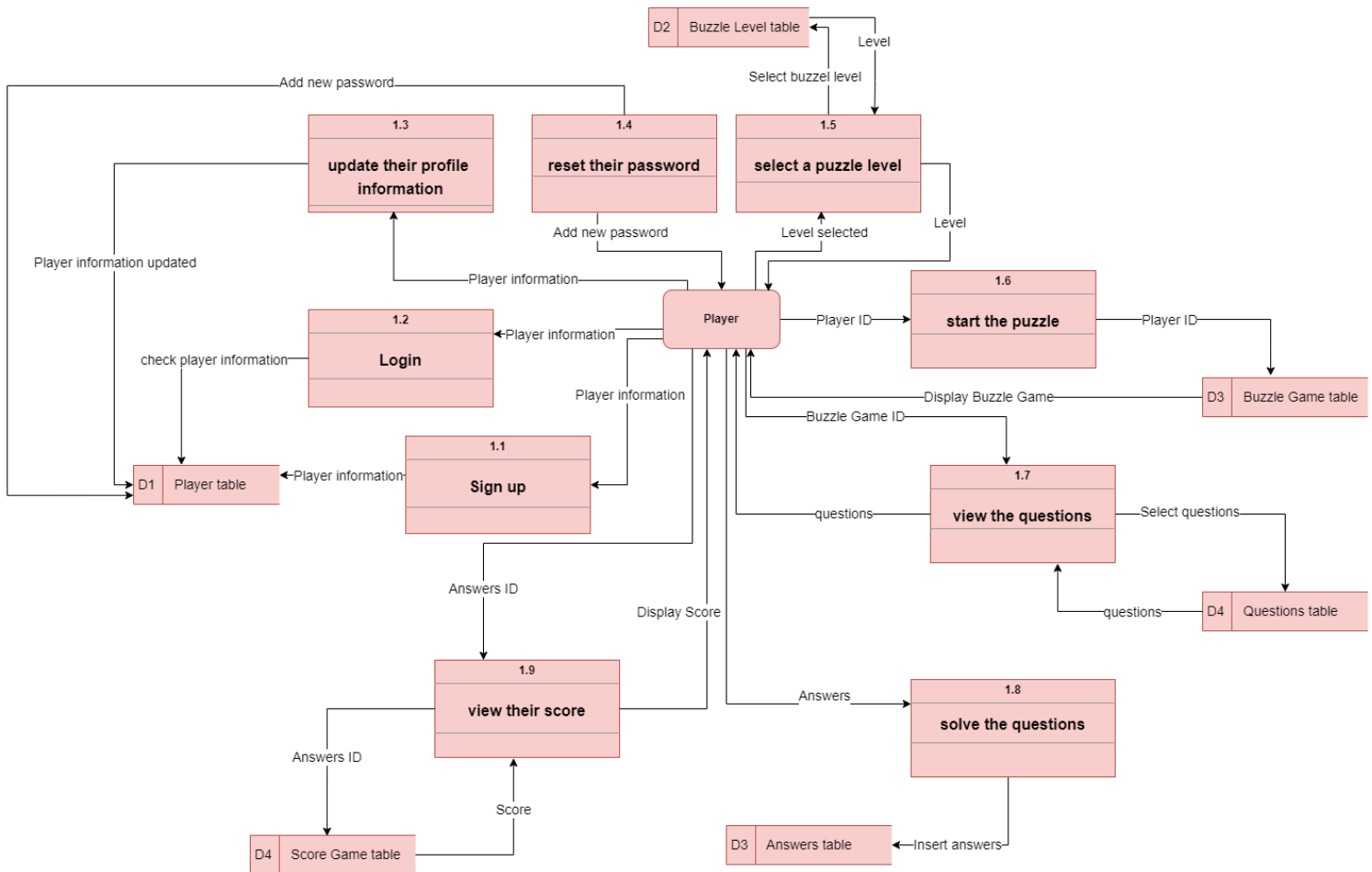


Figure 3. 9: Context Flow Diagrams

3.5 Data Flow Diagrams for Level 1

Data Flow Diagrams for Level 1 as shown in Figure 3.10.

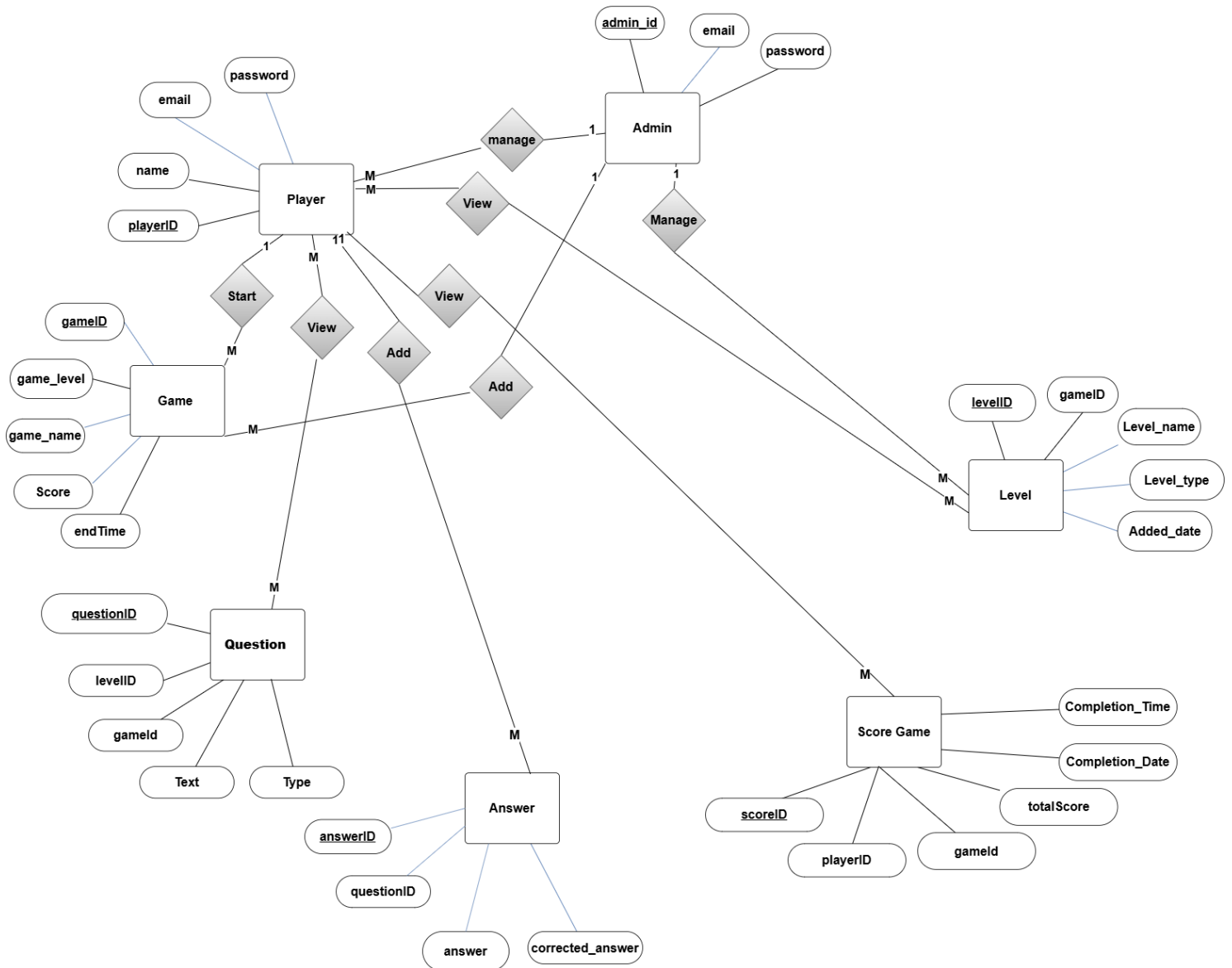


3.6 Database Design

Database design is a set of processes that facilitate the regular creation, development, implementation, and maintenance of an organization's data management systems. A well-designed database is easier to maintain, improves data consistency, and reduces costs on disk storage space. A database designer determines how data elements are related and what data should be stored.

3.6.1 ER Diagram

An entity-relationship (ER) diagram is a graphical representation used in database design and modeling to illustrate the structure of a database system. It visually depicts the entities (objects or concepts) in a database, the attributes (properties) associated with those entities, and the relationships (associations) between the entities. The following figure 3.11 shows the ER diagram of the proposed system.



3.6.2 Database Tables

A data dictionary provides a comprehensive description of the data elements used in the system. It defines the data attributes, their properties, and their relationships.

Table 1: Game Developer

Table 3. 1: Table 1: game developer

Column Name	Data Type	Constraints	Description
Gamedeveloper_ID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the admin
Email	VARCHAR(15)	NOT NULL	Admin's email address
Password	VARCHAR(15)	NOT NULL	Admin's password

Table 2: Player

Table 3. 2: Table 2: Player

Column Name	Data Type	Constraints	Description
playerID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the player
gamedeveloper_id	INT	FOREIGN KEY REFERENCES Game Developer(playerID)	Game Developer ID associated with the Game Developer
Name	VARCHAR(15)	NOT NULL	Player's name
Email	VARCHAR(15)	NOT NULL	Player's email address
password	VARCHAR(15)	NOT NULL	Player's password

Table 3: Game*Table 3.3: Table 3: Puzzle Game*

Column Name	Data Type	Constraints	Description
gameID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the game
game_level	INT	NOT NULL	Game level
game_name	Varchar	NOT NULL	Game name
Score	INT	NOT NULL	Score
endTime	DATETIME	NOT NULL	Game start time

Table 4: Level*Table 3.4: Table 4: Puzzle Level*

Column Name	Data Type	Constraints	Description
levelID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the puzzle level
gamedeveloper_id	INT	FOREIGN KEY REFERENCES Game Developer(playerID)	Game Developer ID associated with the Game Developer
Level_name	VARCHAR(15)	NOT NULL	Level name
Level_type	VARCHAR(15)	NOT NULL	Level type
Added_date	DATETIME	NOT NULL	Added date

Table 5: Question*Table 3.5: Table 5: Question*

Column Name	Data Type	Constraints	Description
questionID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the question
gameID	INT	FOREIGN KEY REFERENCES Puzzle Game(puzzleID)	Puzzle Game ID associated with Puzzle Game
levelID	INT	FOREIGN KEY REFERENCES PuzzleLevel(levelID)	Puzzle level ID associated with the question
Text	TEXT	NOT NULL	Question text
Type	VARCHAR(15)	NOT NULL	Type

Table 6: Answer*Table 3.6: Table 6: Answer*

Column Name	Data Type	Constraints	Description
answerID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the answer
questionID	INT	FOREIGN KEY REFERENCES Question(questionID)	Question ID associated with the answer
answer	VARCHAR(15)	NOT NULL	answer
corrected_answer	VARCHAR(15)	NOT NULL	corrected_answer

Table 7: Score Game*Table 3.7: Table 7: Score Game*

Column Name	Data Type	Constraints	Description
scoreID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the score
playerID	INT	FOREIGN KEY REFERENCES Player(playerID)	Player ID associated with the score
gameId	INT	FOREIGN KEY REFERENCES Game(gameId)	Game ID associated with the Score
totalScore	INT	NOT NULL	Total Score for answer questions
Completion_Date	Date	NOT NULL	Completion Date
Completion_Time	Time	NOT NULL	Completion Time

3.7 Sample interface design



Figure 3. 12: logo of our game

Welcome in

Email:

Password:

Sign In

Home

Dont have an account? [Sign Up](#)



Figure 3. 10: log in page

Welcome in

User Name:

Email:

Password:

Password again:

Sign Up

Home



Figure 3. 11: sign in page

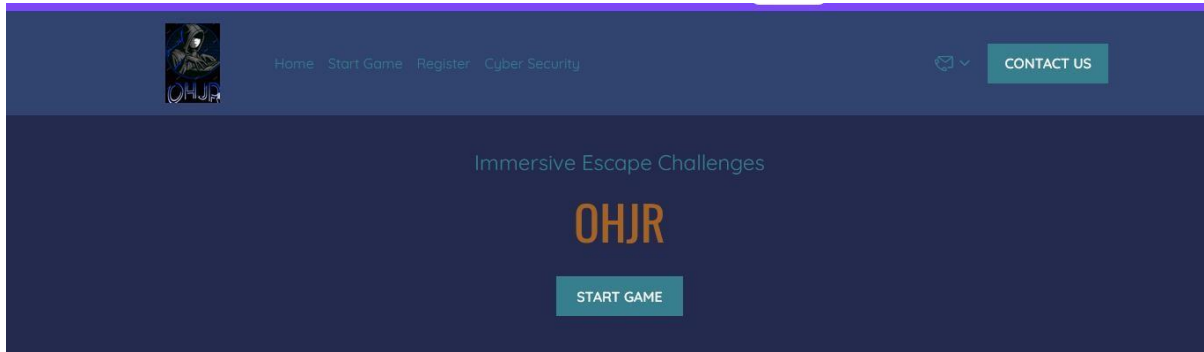


Figure 3. 12: home page in our website



Figure 3. 13: home page

Escape Room Game

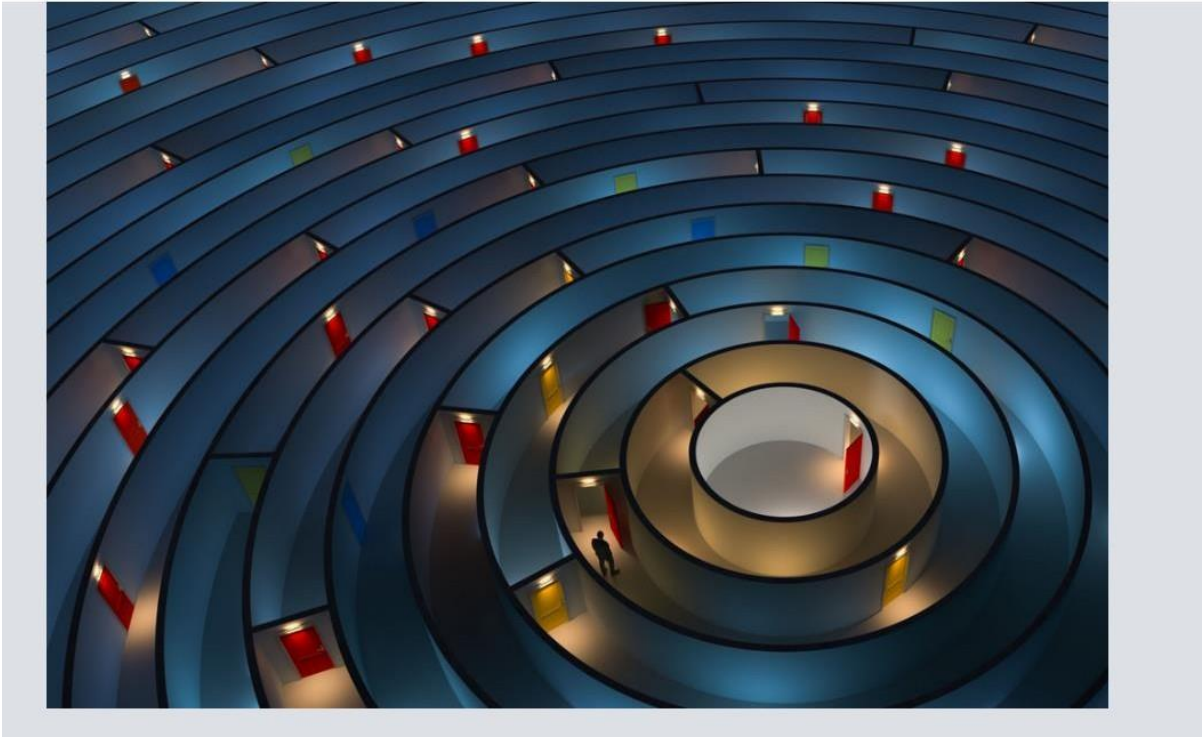


Figure 3. 14: Example of room

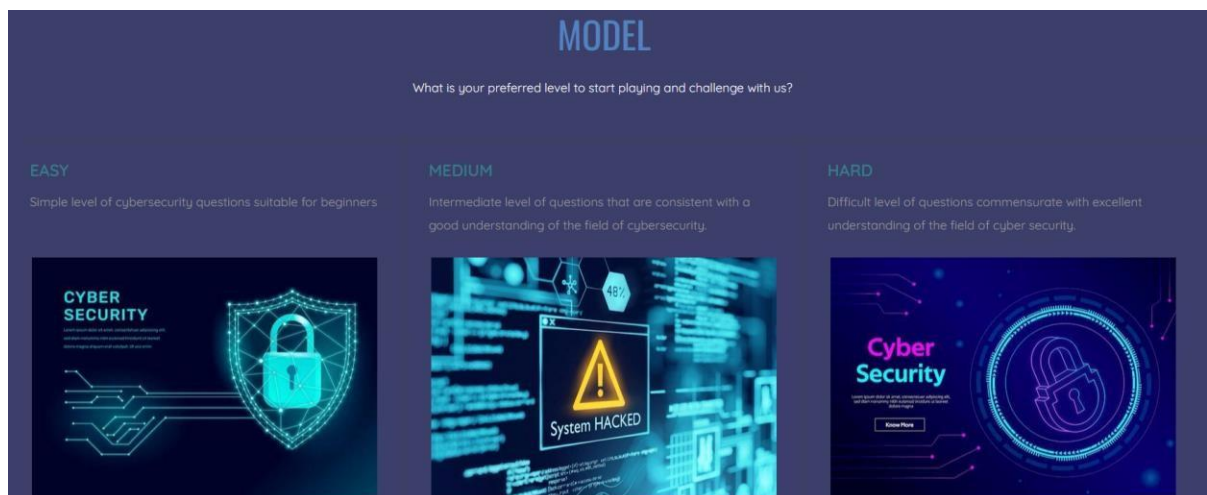


Figure 3. 18: design the model of escape room



Figure 3.19 start game

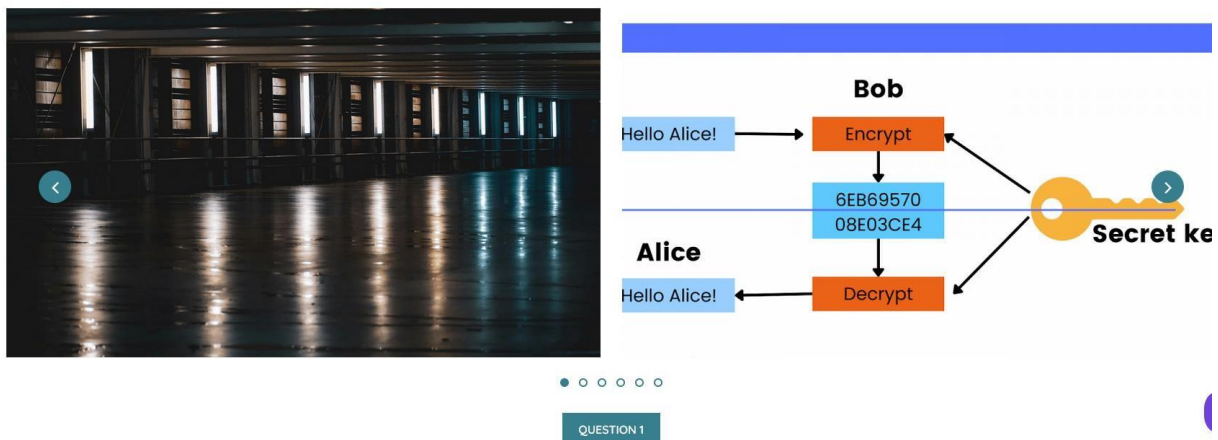


Figure 3.20: Example 1 of question

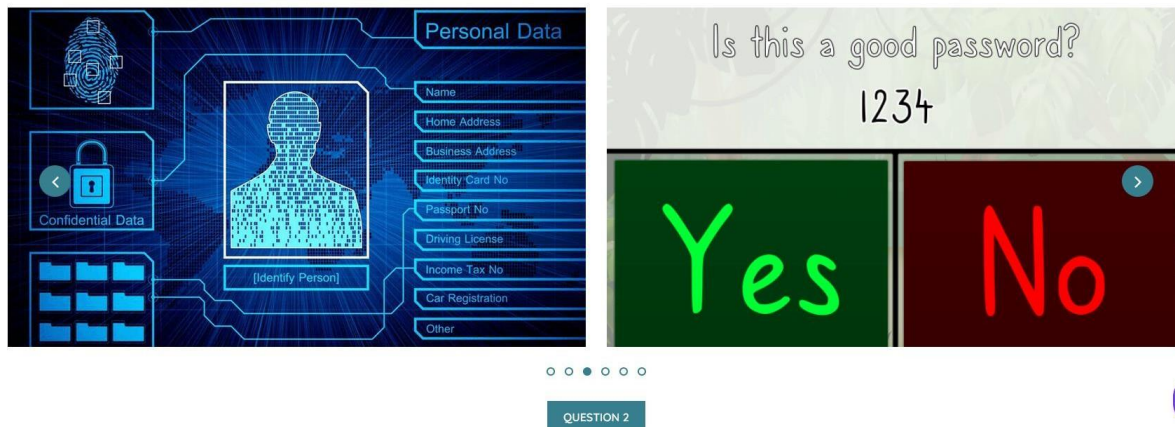


Figure 3. 21: Example 2 of question



Figure 3.22: about the escape room



Figure 3. 23: about the easy level question

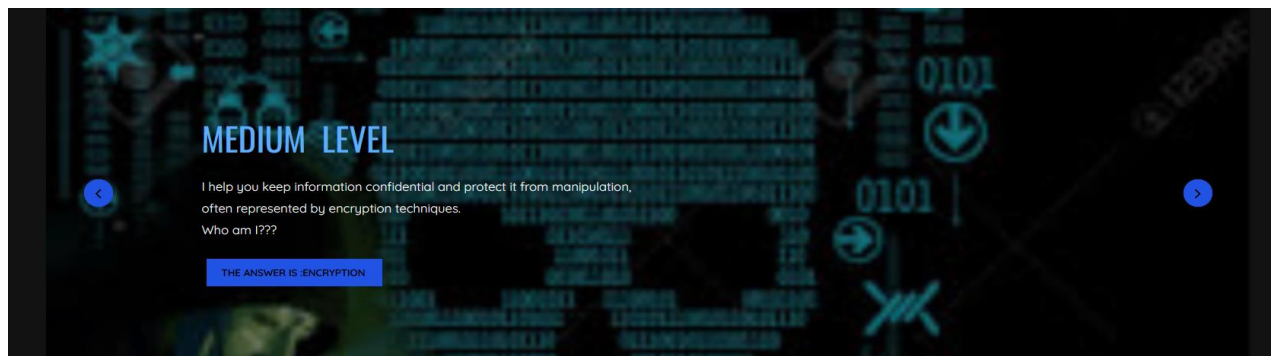


Figure 3. 24: about the medium level question

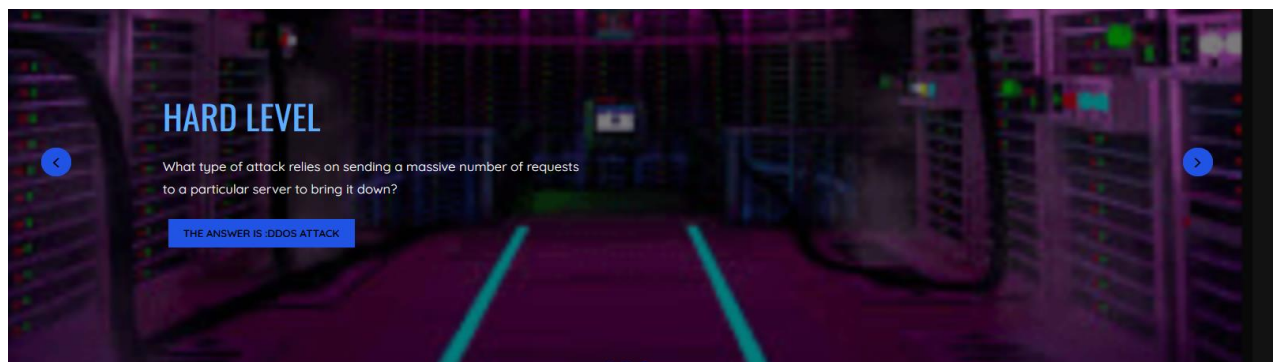


Figure 3. 25: about the hard level question

3.8 Underlying Techniques

Scoring systems in games often use various techniques and considerations to calculate points and evaluate performance. Below are detailed insights into the **Basic Scoring Algorithm** and **Scoring with Time Factor Algorithm**, along with the underlying techniques:

1. Basic Scoring Algorithm (Points-Based Games)

Description:

This algorithm awards points based on player actions, correct answers, or task completions. It's simple and widely used in games where the outcome doesn't depend on time.

Steps:

1. Each correct answer or action earns a fixed number of points.
2. No penalties or minimal deductions for incorrect answers.
3. A total score is calculated by summing the points.

Techniques:

- **Incremental Point System:**
 - Use a counter that increments with each correct answer (e.g., `score += pointsPerQuestion`).
- **Weighting System:**
 - Assign different weights or points to questions based on difficulty (e.g., easy = 1 point, medium = 2 points, hard = 3 points).
- **Event-Driven Scoring:**
 - Points are awarded based on specific triggers or game events (e.g., `onCorrectAnswer() → addPoints()`).

Applications:

- Trivia games.
- Multiple-choice puzzles.
- Games where only accuracy is prioritized.

Pros:

- Simplicity and ease of implementation.
- Clear and intuitive for players.

Cons:

- Does not consider time or strategy, potentially reducing competitiveness.

2. Scoring with Time Factor Algorithm

Description:

Incorporates time as a critical factor in determining the score. Faster responses yield higher scores, rewarding both accuracy and speed.

Steps:

1. Set a maximum score for each question or task.
2. Deduct points based on the time taken to respond.
3. Calculate final score
4. Apply penalties for incorrect answers, if needed.

Techniques:

- **Dynamic Scoring:**
 - Continuously update the score as the player takes time (e.g., $\text{real-time score} = \text{maxScore} - (\text{elapsedTime} * \text{penaltyRate})$).
- **Penalty-Based Deduction:**
 - Deduct a fixed penalty per second (e.g., score -= 1 point per second after 10 seconds).
- **Exponential Decay Function:**
 - Reduce score exponentially based on time, ensuring a significant drop for slower responses.
- **Threshold Time Scoring:**
 - Define time intervals with different scores (e.g., answering within 5 seconds = 10 points, 6–10 seconds = 7 points).

Applications:

- Real-time strategy games.
- Quiz or puzzle games with a competitive edge.
- Timed scenarios like escape rooms or speed challenges.

Pros:

- Encourages quick thinking and decision-making.
- Adds a competitive element to gameplay.

Cons:

- May penalize thoughtful players who take longer to answer.
- Complex to implement compared to basic scoring.

Advanced Scoring Techniques (Optional Enhancements)**1. Streak Bonuses:**

- Award extra points for consecutive correct answers (e.g., 3 correct answers in a row = bonus points).

2. Adaptive Difficulty Scoring:

- Dynamically adjust scores based on player performance or question difficulty.

3. Penalty for Incorrect Answers:

- Deduct points for wrong answers or skipping questions.

4. Accuracy vs. Time Trade-off:

- Create a balance where accuracy is rewarded more than speed for specific puzzles.

Key Considerations:

- **Game Design Goals:** Align scoring mechanisms with game objectives (e.g., encourage speed or careful thought).
- **Player Motivation:** Ensure the scoring system engages players without undue frustration.
- **Fairness:** Avoid overly punishing slower players or those who guess incorrectly.

3.9 Summary

In this chapter, we have covered the system analysis interaction, as well as the UML use case diagram, class diagram, sequence diagram, and activity diagram for our application. We have also discussed the context and data flow diagrams. In addition, the database design, ER diagram, and database tables for our proposed system have been designed. The sample interface design has also been included. Furthermore, the techniques and algorithms of our application are explained in detail at the end of this chapter.

Chapter 4: System Implementation

4.1 Introduction

This chapter examines the project's database design, the development of user-friendly system interfaces, and the creation of the core codebase. Key technologies used in the implementation are vital in optimizing data management, interface design, and backend development, ensuring the project's success.

4.2 Software Tools Used in Implementation

In this section, we describe all the technological tools used to complete the project, which are the software tools for developing this project.

- Asana: For project management and puzzle organization.
- Canva: for designing intuitive user interfaces
- My SQL Server: for creating and store data such as (player data. and game progress)
- Visual studio code: for Web Development Frameworks:
 3. Front end for interfaces the escape room game. we will use (HTML. CSS. javascript, React)
 4. A back end for real-time interactions is required we will use (PHP)

4.3 Database Implementation

Database design is a critical aspect of software development that involves creating the structure, organization, and architecture of a database system. It defines how data is stored, accessed, and managed within a database, ensuring efficient data retrieval, integrity, and security. Here are some key components and considerations in the database design process:

1. **Data Modeling:** Data modeling is the initial step in database design, where you define the data entities (objects) and their relationships. This is often done using techniques like Entity-Relationship Diagrams (ERDs) or Unified Modeling Language (UML) diagrams.

2. **Schema Design:** Once the data model is established, you proceed to schema design. A schema defines the structure of tables, columns, data types, constraints, and indexes in the database. It's essential to choose appropriate data types and constraints to ensure data accuracy and integrity.
3. **Normalization:** Normalization is the process of organizing data in a database to eliminate redundancy and reduce data anomalies. It involves breaking down tables into smaller, related tables and establishing relationships between them.
4. **Indexing:** Indexes are used to speed up data retrieval operations. Designing effective indexes is crucial for optimizing query performance. You need to determine which columns to index and when to use different types of indexes (e.g., B-tree, hash, or full-text indexes).
5. **Data Integrity and Constraints:** Implementing data integrity rules and constraints is vital to maintaining data quality and accuracy. This includes primary keys, foreign keys, unique constraints, and check constraints.
6. **Security:** Database security measures, such as user roles, permissions, and encryption, must be considered to protect sensitive data from unauthorized access and breaches.
7. **Performance Tuning:** Database design should also focus on performance optimization. Techniques like denormalization, caching, and query optimization can be used to enhance database performance.

4.4 Database Design

A database is a structured collection of information that can be accessed, managed, updated, and deleted. It organizes data into tables, rows, and columns, making it easier to query for relevant information. Additionally, data is often indexed to improve accessibility. The following sections will outline the database scheme.

4.4.1 Users Table

The user table as shown in Figure 4.1.

Server: 127.0.0.1 » Database: escape_game » Table: users

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)
[Tracking](#)
[Trig](#)

[Table structure](#)
[Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	username	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	email	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	password	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐ 5	type	int(11)			No	0			Change Drop More

☐ Check all
 With selected:
 [Browse](#)
[Change](#)
[Drop](#)
[Primary](#)
[Unique](#)
[Index](#)
[Spatial](#)
[Fulltext](#)

[Add to central columns](#)
[Remove from central columns](#)

Figure 4. 1: Users Table

4.4.2 Score Game Table

The score game table as shown in Figure 4.2.

Server: 127.0.0.1 » Database: escape_game » Table: scoregame

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)
[Tracking](#)
[Trig](#)

[Table structure](#)
[Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	playerID	int(11)			Yes	NULL			Change Drop More
☐ 3	gameID	int(11)			Yes	NULL			Change Drop More
☐ 4	totalScore	int(11)			Yes	NULL			Change Drop More
☐ 5	completionTime	int(11)			Yes	NULL			Change Drop More
☐ 6	completionDate	timestamp			No	current_timestamp()			Change Drop More

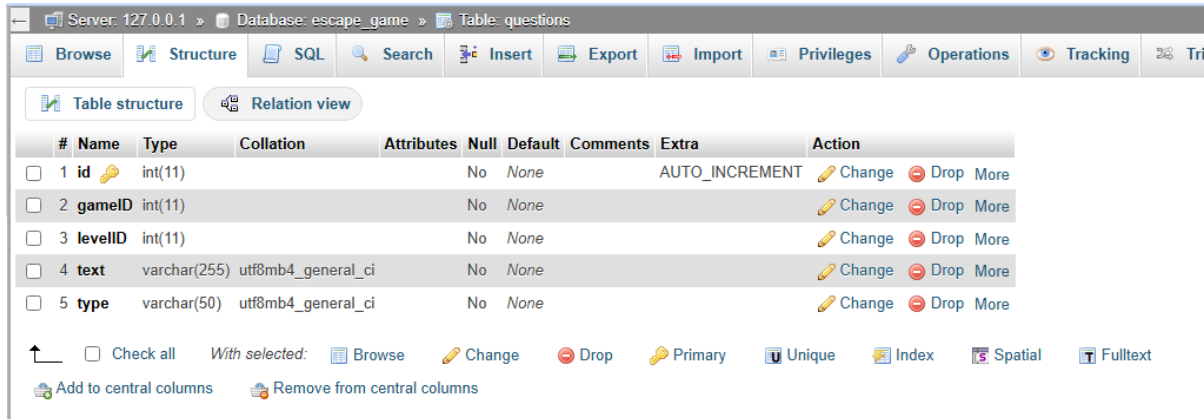
☐ Check all
 With selected:
 [Browse](#)
[Change](#)
[Drop](#)
[Primary](#)
[Unique](#)
[Index](#)
[Spatial](#)
[Fulltext](#)

[Add to central columns](#)
[Remove from central columns](#)

Figure 4. 2: Score Game Table

4.4.3 Questions Table

The questions table as shown in Figure 4.3.

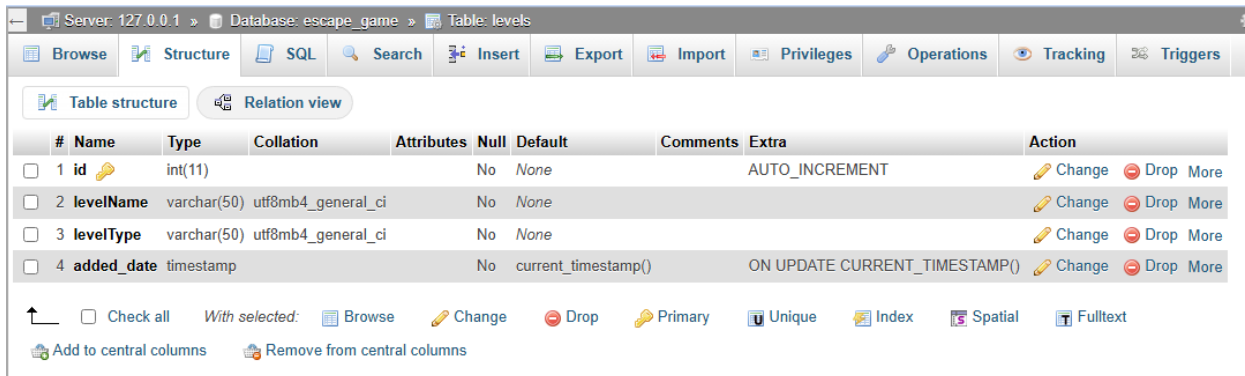


#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 gameId	int(11)			No	None			Change Drop More
☐	3 levelID	int(11)			No	None			Change Drop More
☐	4 text	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐	5 type	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More

Figure 4. 3: Questions Table

4.4.4 Levels Table

The levels table as shown in Figure 4.4.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 levelName	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 levelType	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 added_date	timestamp			No	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP()	Change Drop More

Figure 4. 4: Levels Table

4.4.5 Games Table

The games table as shown in Figure 4.5.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	gameLevel	int(11)			No	None			Change Drop More
3	gameName	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
4	endTime	int(11)			No	None			Change Drop More
5	score	int(11)			No	None			Change Drop More

Figure 4. 5: Games Table

4.4.6 Answers Table

The answers table as shown in Figure 4.6.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	questionID	int(11)			No	None			Change Drop More
3	answer	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
4	correctAnswer	int(11)			No	None			Change Drop More

Figure 4. 6: Answers Table

4.5 Interface Design

The user interface design stage is very important because it becomes the foundation of the interaction of users with our system. It has a great impact on user acceptance of the designed site as it acts as the chief point of attraction through attractive colors, interactive pages, and handsome-looking icons and buttons. Below, we are going to illustrate the user interfaces achieved along with their short descriptions.

4.5.1 Home Interface

As shown in Figure 4.7, The Home Interface is the main landing page, introducing the game and navigation options. Built with HTML, CSS, and JavaScript.



Figure 4. 7: Home Interface

4.5.2 Sign-up Interface

As shown in Figure 4.8, Users can create an account by entering credentials. JavaScript validates input, while PHP processes data and stores it in MySQL Server.

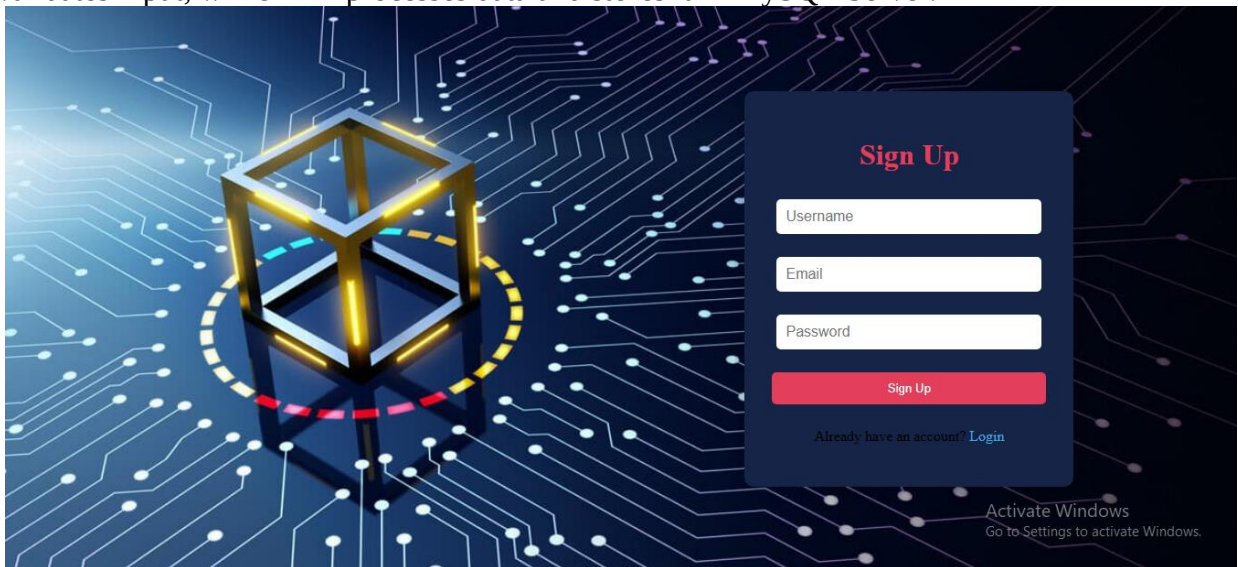


Figure 4. 8: Sign-up Interface

4.5.3 Login Interface

As shown in Figure 4.9, Registered users log in with email and password. JavaScript handles validation, and PHP authenticates credentials using MySQL Server.

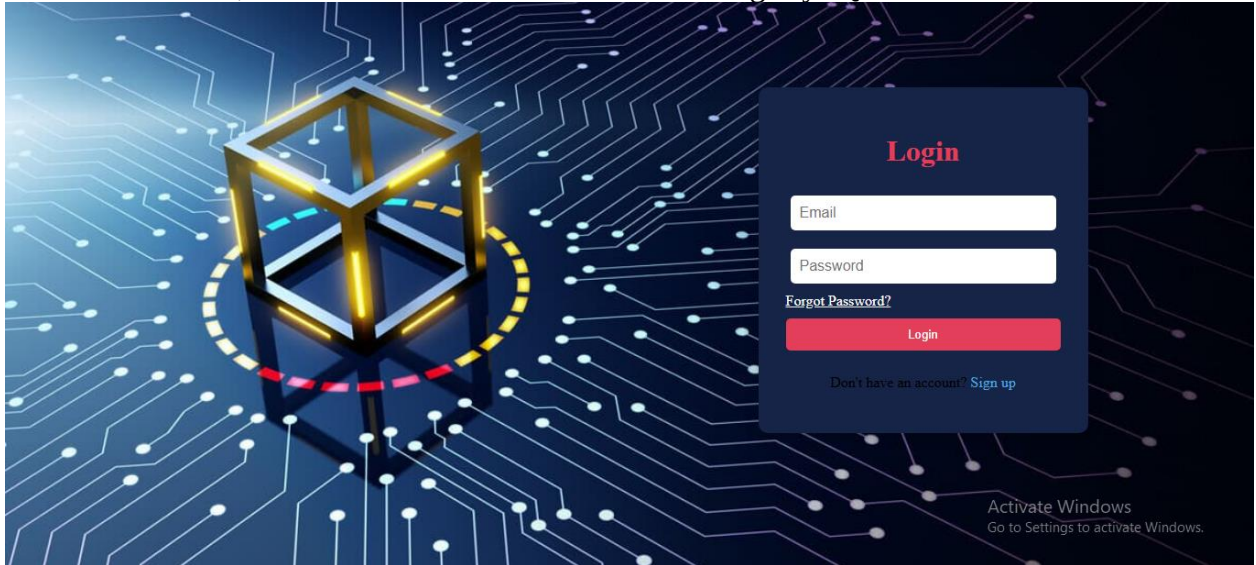


Figure 4. 9: Login Interface

4.5.4 Levels Interface

As shown in Figure 4.10, Players select difficulty levels (easy, medium, hard). Built with React for interactivity, with CSS and JavaScript for design and logic.

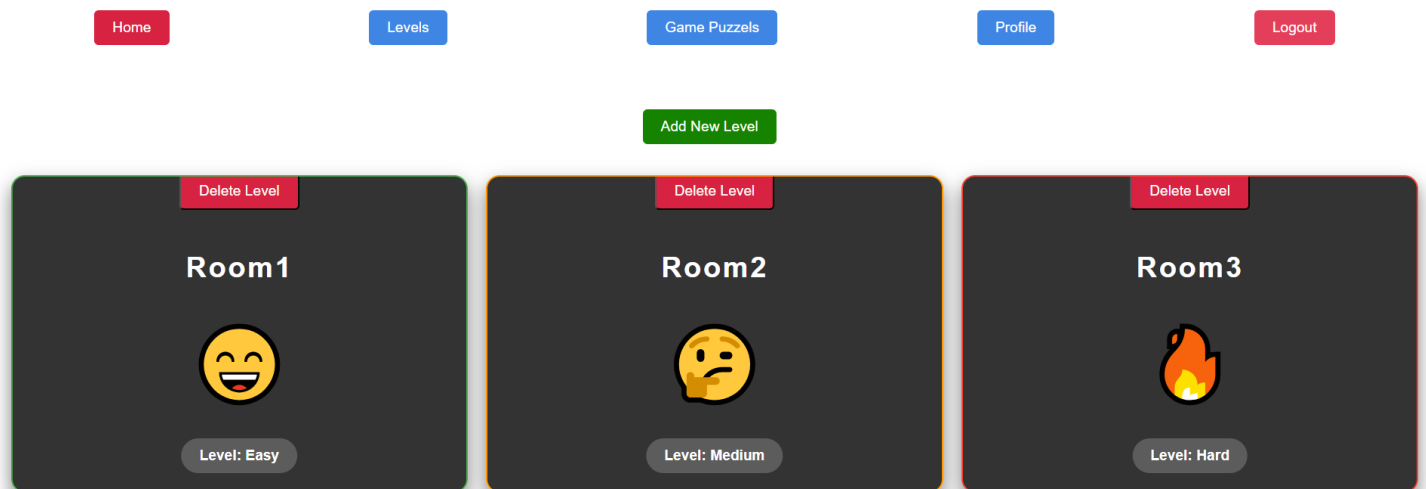


Figure 4. 10: Levels Interface

4.5.5 Room 1 Interface

As shown in Figure 4.11, The first level features basic cybersecurity puzzles. React provides smooth navigation, JavaScript handles game logic, and MySQL Server saves progress.

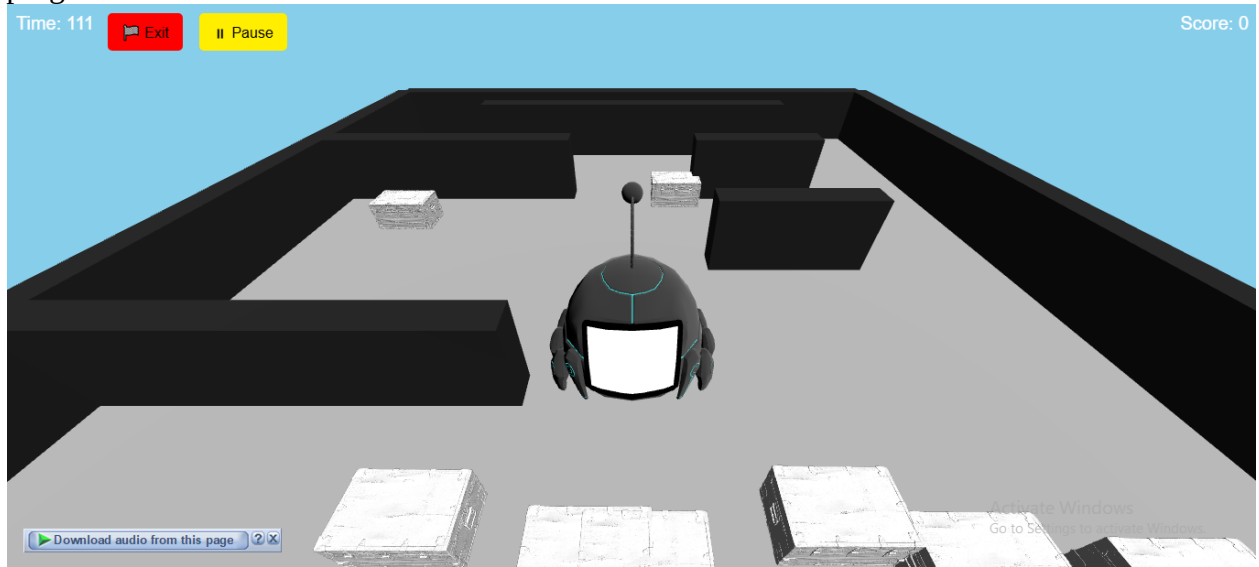


Figure 4. 11: Room 1 Interface

4.5.6 Room 2 Interface

As shown in Figure 4.12, Mid-level puzzles with real-time interaction. React enhances engagement, JavaScript manages game mechanics, and PHP connects with MySQL Server.

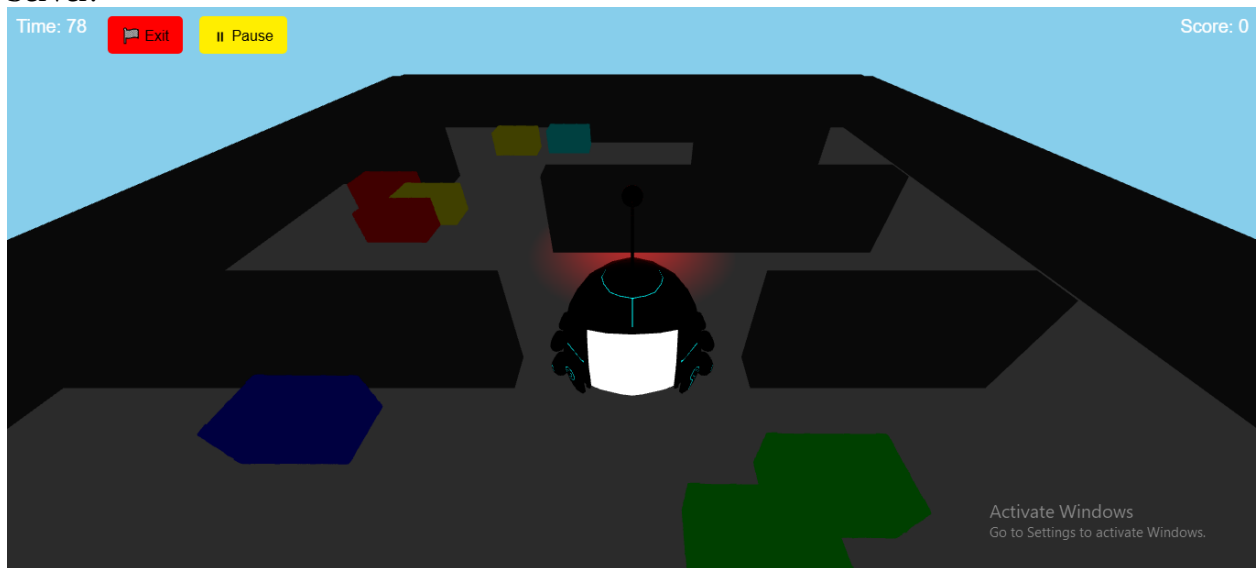


Figure 4. 12: Room 2 Interface

4.5.7 Room 3 Interface

As shown in Figure 4.13, The final and most challenging level. React powers the UI, JavaScript controls logic, PHP enables real-time interaction, and MySQL Server stores progress.

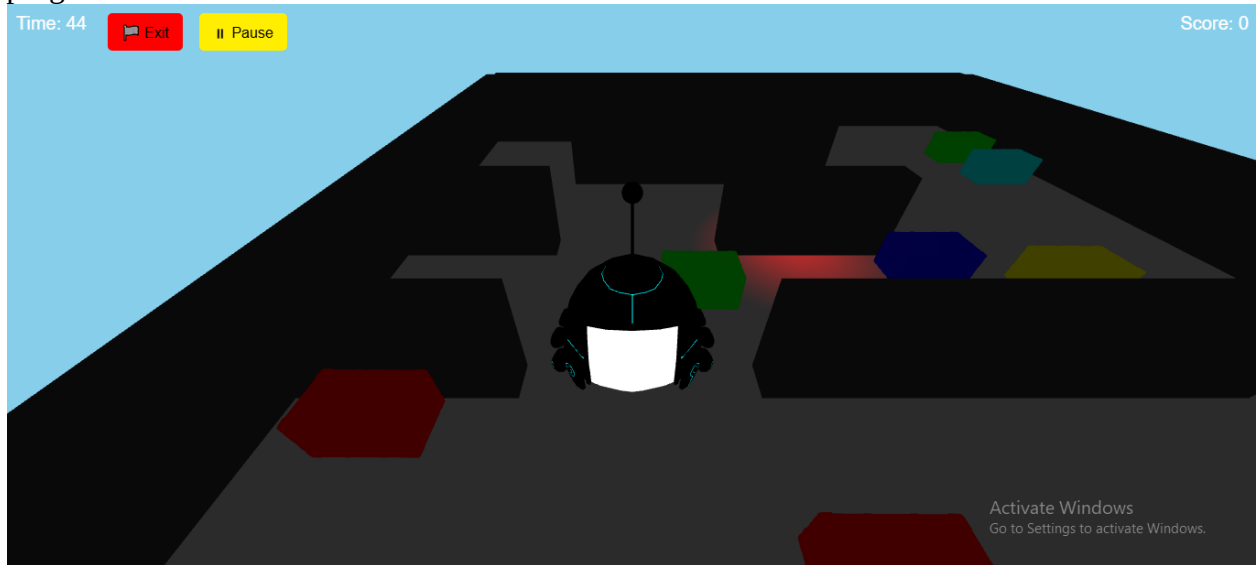


Figure 4. 13: Room 3 Interface

4.5.8 Add Level for Admin Interface

As shown in Figure 4.14, this interface lets admins create new game levels by entering the name, difficulty, and description. Built with HTML, PHP, and JavaScript, it stores data in MySQL after validation.

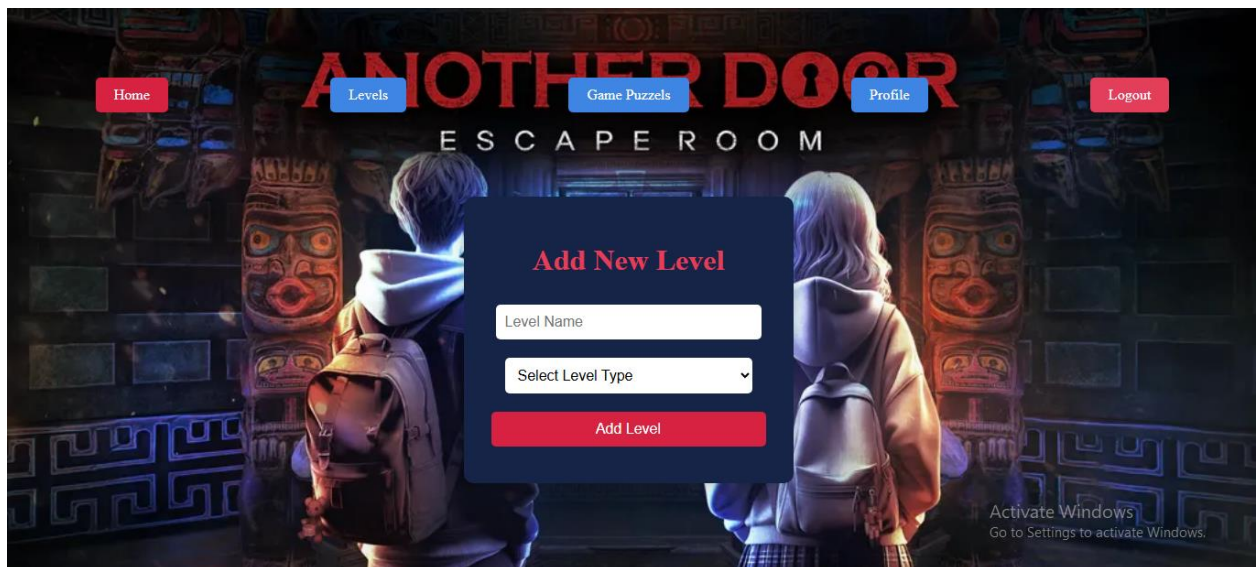


Figure 4. 14: Add Level for Admin Interface

4.5.9 Admin Add Puzzle Interface

As shown in Figure 4.15, this interface allows admins to add puzzles with questions, hints, and answers. JavaScript handles validation, while PHP and MySQL manage data storage.

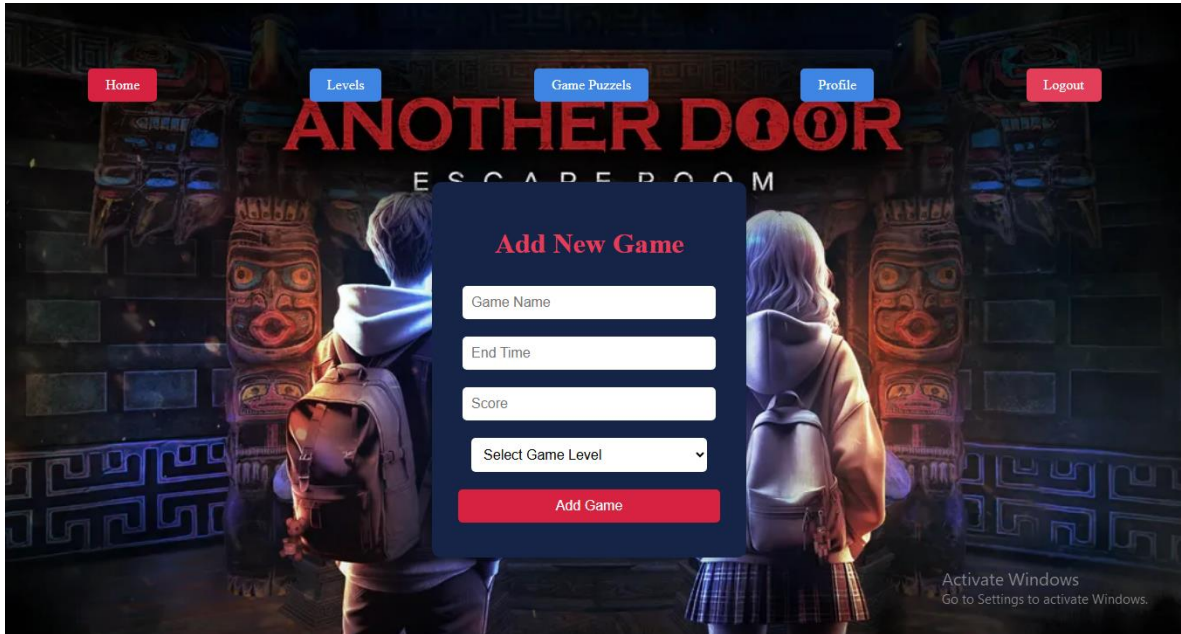


Figure 4. 15: Admin Add Puzzle Interface

4.5.10 Admin Levels List Interface

As shown in Figure 4.16, this interface displays all game levels with options to edit or delete. It uses PHP and JavaScript to fetch and manage data from MySQL.

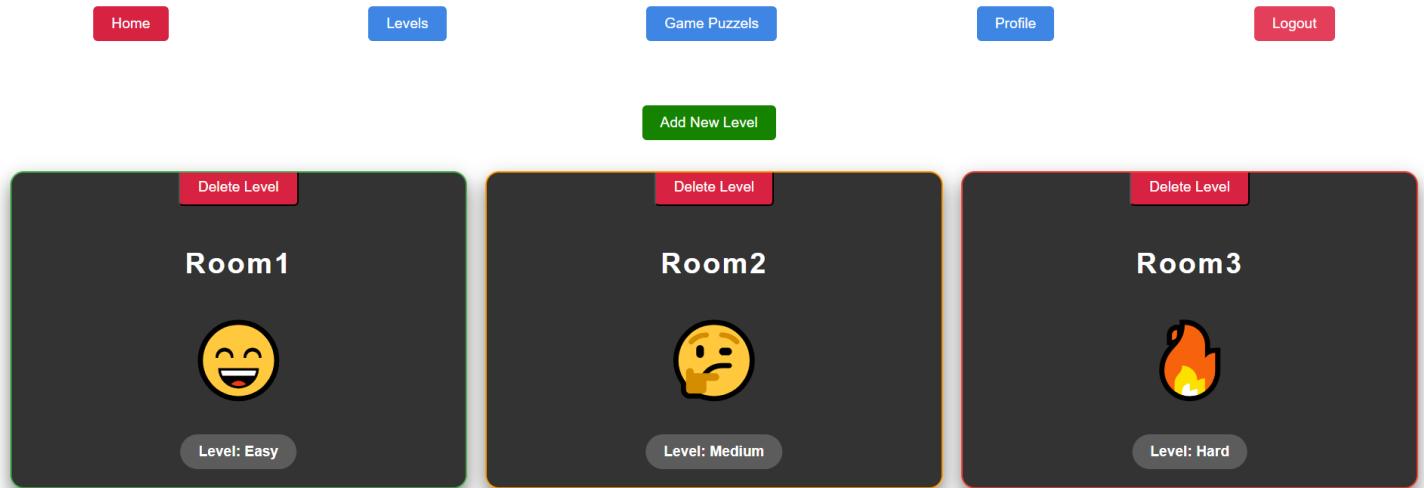


Figure 4. 16: Admin Levels List Interface

4.5.11 Admin Puzzles List Interface

As shown in Figure 4.17, this interface lists all puzzles with their levels and provides edit/delete options. PHP and JavaScript handle data operations and interaction with the MySQL Server.



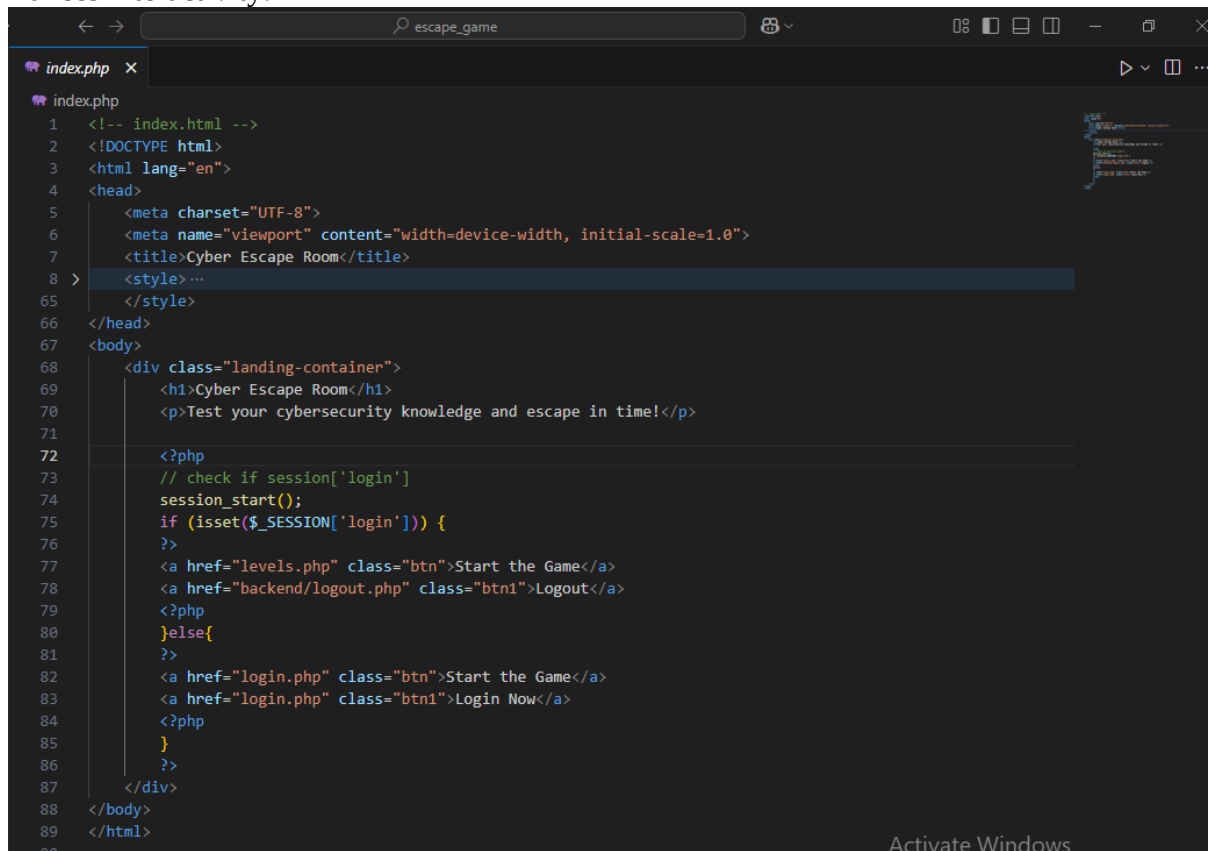
Figure 4. 17: Admin Puzzles List Interface

4.6 Sample Code

This section presents the core code implementations for various game interfaces. Each code segment is developed using HTML, CSS, JavaScript, React, PHP, and MySQL Server, ensuring seamless functionality, interactive gameplay, and secure data management.

4.6.1 Home Code

As shown in Figure 4.18, The Home Code structures the main landing page using HTML, CSS, and JavaScript, ensuring smooth navigation and responsiveness. React enhances interactivity.



```
index.php
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Cyber Escape Room</title>
8   <style>...
65 </style>
66 </head>
67 <body>
68   <div class="landing-container">
69     <h1>Cyber Escape Room</h1>
70     <p>Test your cybersecurity knowledge and escape in time!</p>
71
72   <?php
73     // check if session['login']
74     session_start();
75     if (isset($_SESSION['login'])) {
76     }
77     <a href="levels.php" class="btn">Start the Game</a>
78     <a href="backend/logout.php" class="btn1">Logout</a>
79   <?php
80   }else{
81   }
82   <a href="login.php" class="btn">Start the Game</a>
83   <a href="login.php" class="btn1">Login Now</a>
84   <?php
85   }
86   ?>
87   </div>
88 </body>
89 </html>
```

Figure 4. 18: Home Code

4.6.2 Sign-up Code

As shown in Figure 4.19, This code handles user registration with HTML forms, JavaScript validation, and PHP backend processing. User data is securely stored in MySQL Server.


```
escape_game

signup.php X
signup.php
1 <!-- signup.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Sign Up</title>
8   <style>...
60 </style>
61 </head>
62 <body>
63   <div class="form-container">
64     <h1>Sign Up</h1>
65     <form action="backend/access.php" method="post">
66       <input type="text" name="username" placeholder="Username" required>
67       <input type="email" name="email" placeholder="Email" required>
68       <input type="password" name="password" placeholder="Password" required>
69       <button type="submit" name="signup" class="btn">Sign Up</button>
70       <p>Already have an account? <a href="login.php">Login</a></p>
71     </form>
72   </div>
73 </body>
74 </html>
75
```

Figure 4. 19: Sign-up Code

4.6.3 Login Code

As shown in Figure 4.20, The login system verifies user credentials using JavaScript for validation, PHP for authentication, and MySQL Server for database management.

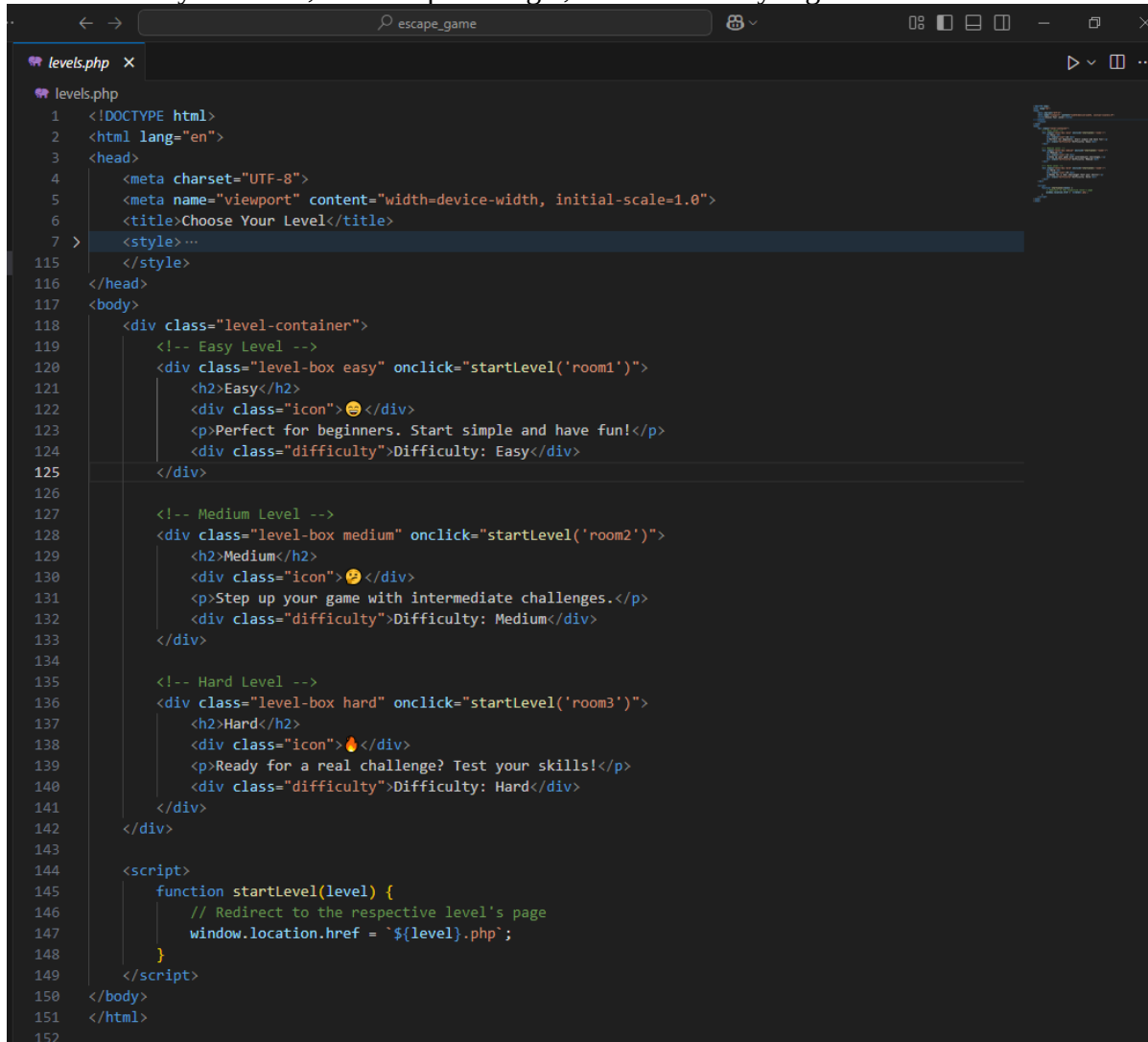
```
escape_game

login.php X
login.php
1 <!-- login.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Login</title>
8   <style>...
60 </style>
61 </head>
62 <body>
63   <div class="form-container">
64     <h1>Login</h1>
65     <form action="backend/access.php" method="post">
66       <input type="text" name="email" placeholder="Email" required>
67       <input type="password" name="password" placeholder="Password" required>
68       <button type="submit" name="login" class="btn">Login</button>
69       <p>Don't have an account? <a href="signup.php">Sign up</a></p>
70     </form>
71   </div>
72 </body>
73 </html>
74
```

Figure 4. 20: Login Code

4.6.4 Levels Code

As shown in Figure 4.21, This code enables players to choose difficulty levels. Built with React for dynamic UI, JavaScript for logic, and CSS for styling.

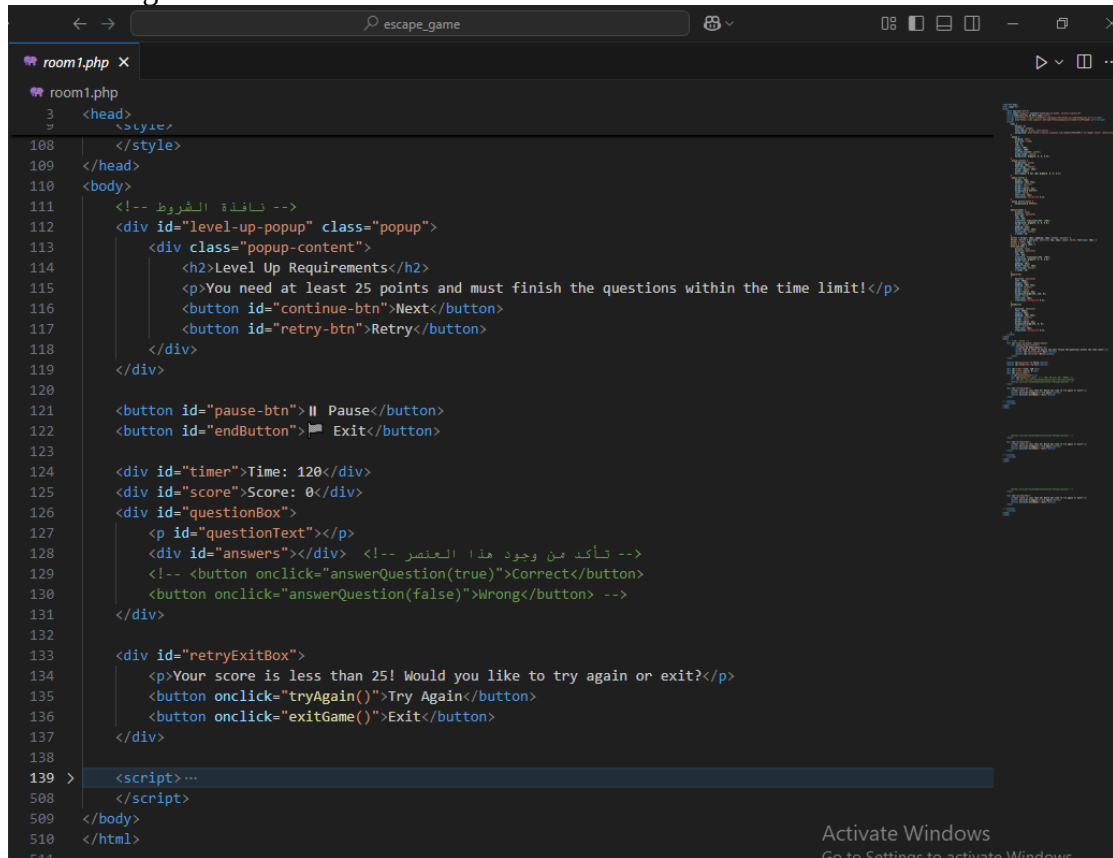


```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Choose Your Level</title>
7   <style>...
115 </style>
116 </head>
117 <body>
118   <div class="level-container">
119     <!-- Easy Level -->
120     <div class="level-box easy" onclick="startLevel('room1')">
121       <h2>Easy</h2>
122       <div class="icon">😊</div>
123       <p>Perfect for beginners. Start simple and have fun!</p>
124       <div class="difficulty">Difficulty: Easy</div>
125     </div>
126
127     <!-- Medium Level -->
128     <div class="level-box medium" onclick="startLevel('room2')">
129       <h2>Medium</h2>
130       <div class="icon">😬</div>
131       <p>Step up your game with intermediate challenges.</p>
132       <div class="difficulty">Difficulty: Medium</div>
133     </div>
134
135     <!-- Hard Level -->
136     <div class="level-box hard" onclick="startLevel('room3')">
137       <h2>Hard</h2>
138       <div class="icon">🔥</div>
139       <p>Ready for a real challenge? Test your skills!</p>
140       <div class="difficulty">Difficulty: Hard</div>
141     </div>
142   </div>
143
144   <script>
145     function startLevel(level) {
146       // Redirect to the respective level's page
147       window.location.href = `${level}.php`;
148     }
149   </script>
150 </body>
151 </html>
152
```

Figure 4. 21: Levels Code

4.6.5 Room 1 Code

As shown in Figure 4.22, Implements beginner-level cybersecurity puzzles using JavaScript for interactions, React for UI updates, and PHP with MySQL Server for progress tracking.

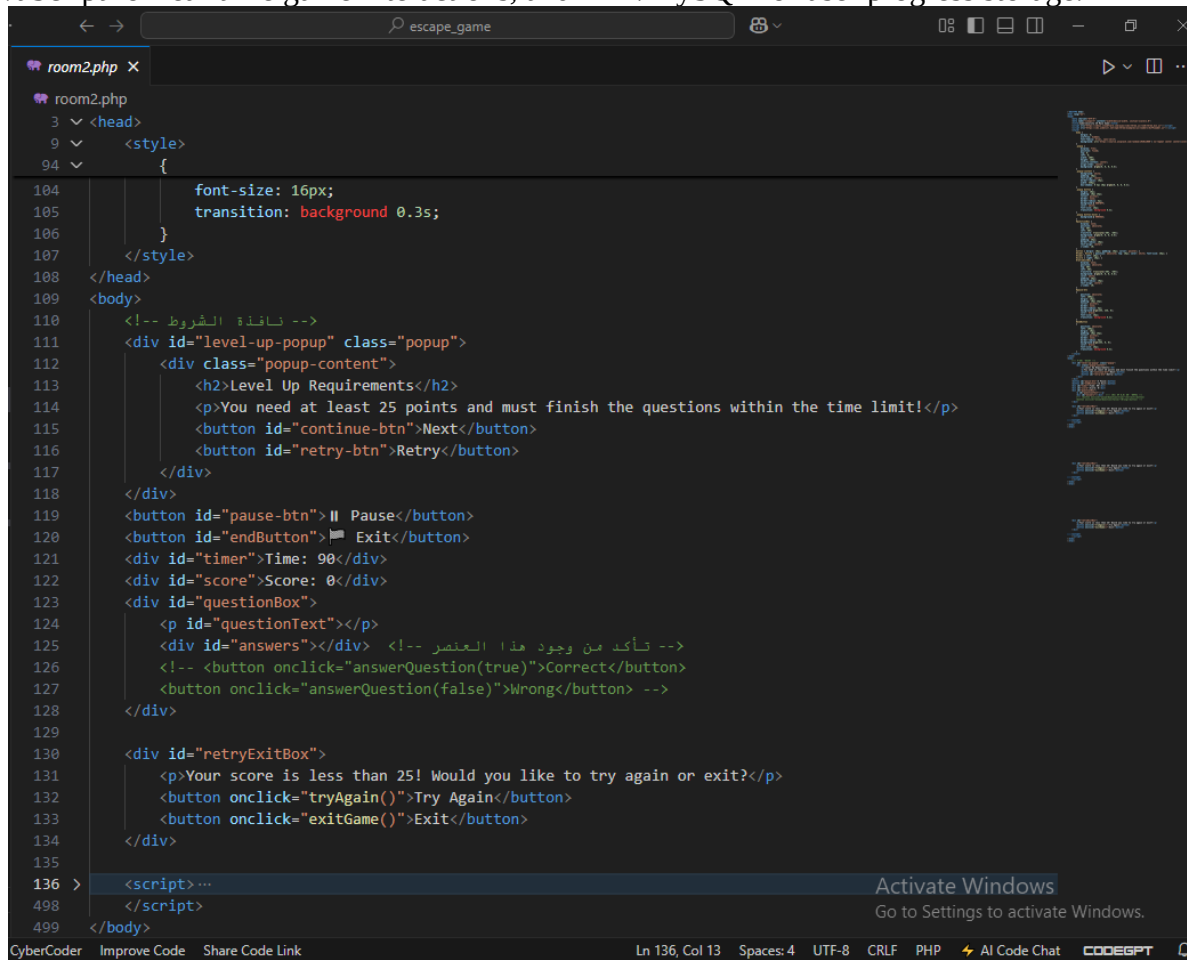


```
108 </style>
109 </head>
110 <body>
111   <!-- نافذة الشروط -->
112   <div id="level-up-popup" class="popup">
113     <div class="popup-content">
114       <h2>Level Up Requirements</h2>
115       <p>You need at least 25 points and must finish the questions within the time limit!</p>
116       <button id="continue-btn">Next</button>
117       <button id="retry-btn">Retry</button>
118     </div>
119   </div>
120
121   <button id="pause-btn">⏸ Pause</button>
122   <button id="endButton">🚪 Exit</button>
123
124   <div id="timer">Time: 120</div>
125   <div id="score">Score: 0</div>
126   <div id="questionBox">
127     <p id="questionText"></p>
128     <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
129     <!-- <button onclick="answerQuestion(true)">Correct</button>
130     <button onclick="answerQuestion(false)">Wrong</button> -->
131   </div>
132
133   <div id="retryExitBox">
134     <p>Your score is less than 25! Would you like to try again or exit?</p>
135     <button onclick="tryAgain()">Try Again</button>
136     <button onclick="exitGame()">Exit</button>
137   </div>
138
139 > <script>...
508 </script>
509 </body>
510 </html>
511
```

Figure 4.22: Room 1 Code

4.6.6 Room 2 Code

As shown in Figure 4.23, Mid-level challenges coded using React for front-end logic, JavaScript for real-time game interactions, and PHP/MySQL for user progress storage.

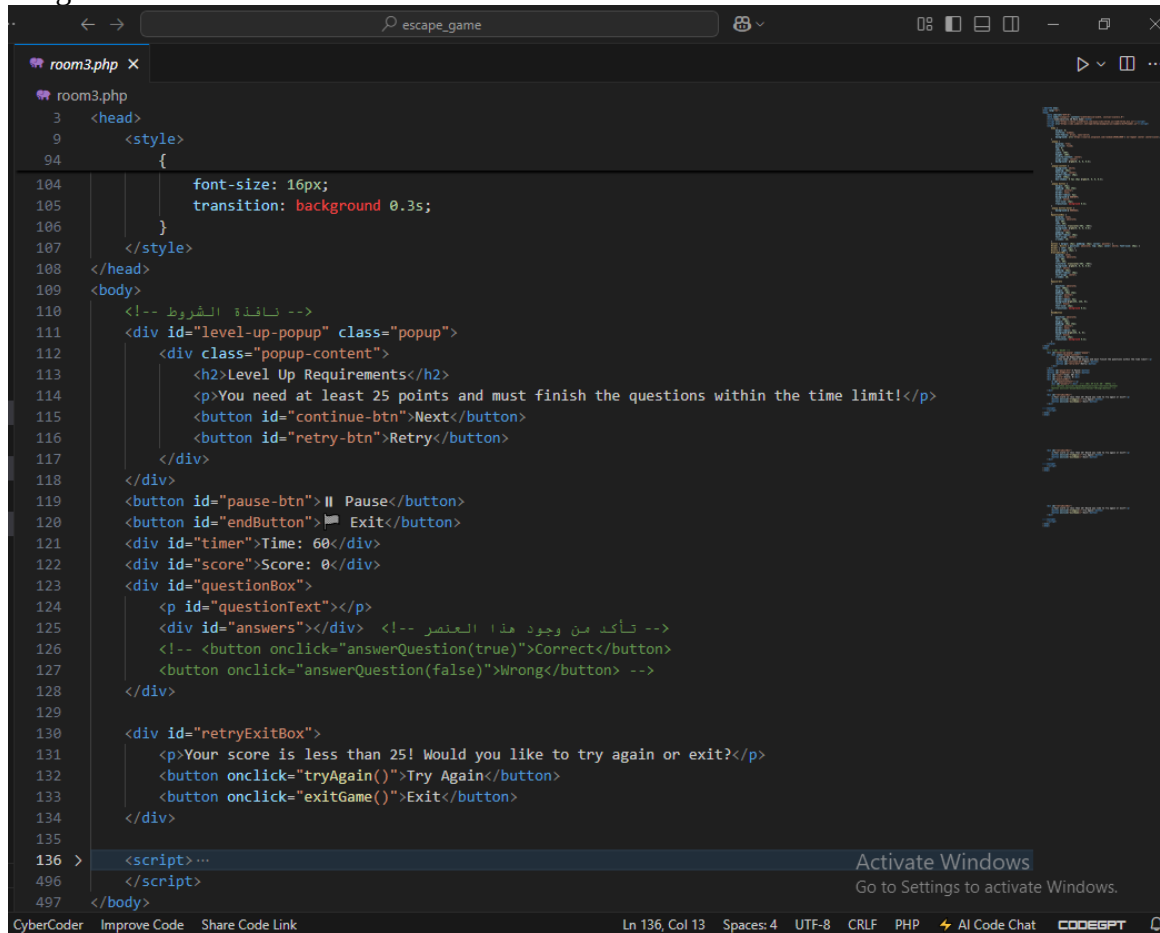


```
room2.php
3 <head>
9 <style>
94 {
104     font-size: 16px;
105     transition: background 0.3s;
106 }
107 </style>
108 </head>
109 <body>
110 <!-- نافذة الشروط -->
111 <div id="level-up-popup" class="popup">
112     <div class="popup-content">
113         <h2>Level Up Requirements</h2>
114         <p>You need at least 25 points and must finish the questions within the time limit!</p>
115         <button id="continue-btn">Next</button>
116         <button id="retry-btn">Retry</button>
117     </div>
118 </div>
119 <button id="pause-btn">⏸ Pause</button>
120 <button id="endButton">🚪 Exit</button>
121 <div id="timer">Time: 90</div>
122 <div id="score">Score: 0</div>
123 <div id="questionBox">
124     <p id="questionText"></p>
125     <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
126     <!-- <button onclick="answerQuestion(true)">Correct</button>
127         <button onclick="answerQuestion(false)">Wrong</button> -->
128 </div>
129
130 <div id="retryExitBox">
131     <p>Your score is less than 25! Would you like to try again or exit?</p>
132     <button onclick="tryAgain()">Try Again</button>
133     <button onclick="exitGame()">Exit</button>
134 </div>
135
136 <script>...
498 </script>
499 </body>
```

Figure 4. 23: Room 2 Code

4.6.7 Room 3 Code

As shown in Figure 4.24, The final and most complex level, integrating React for UI, JavaScript for puzzle mechanics, and PHP with MySQL Server to ensure seamless data handling.

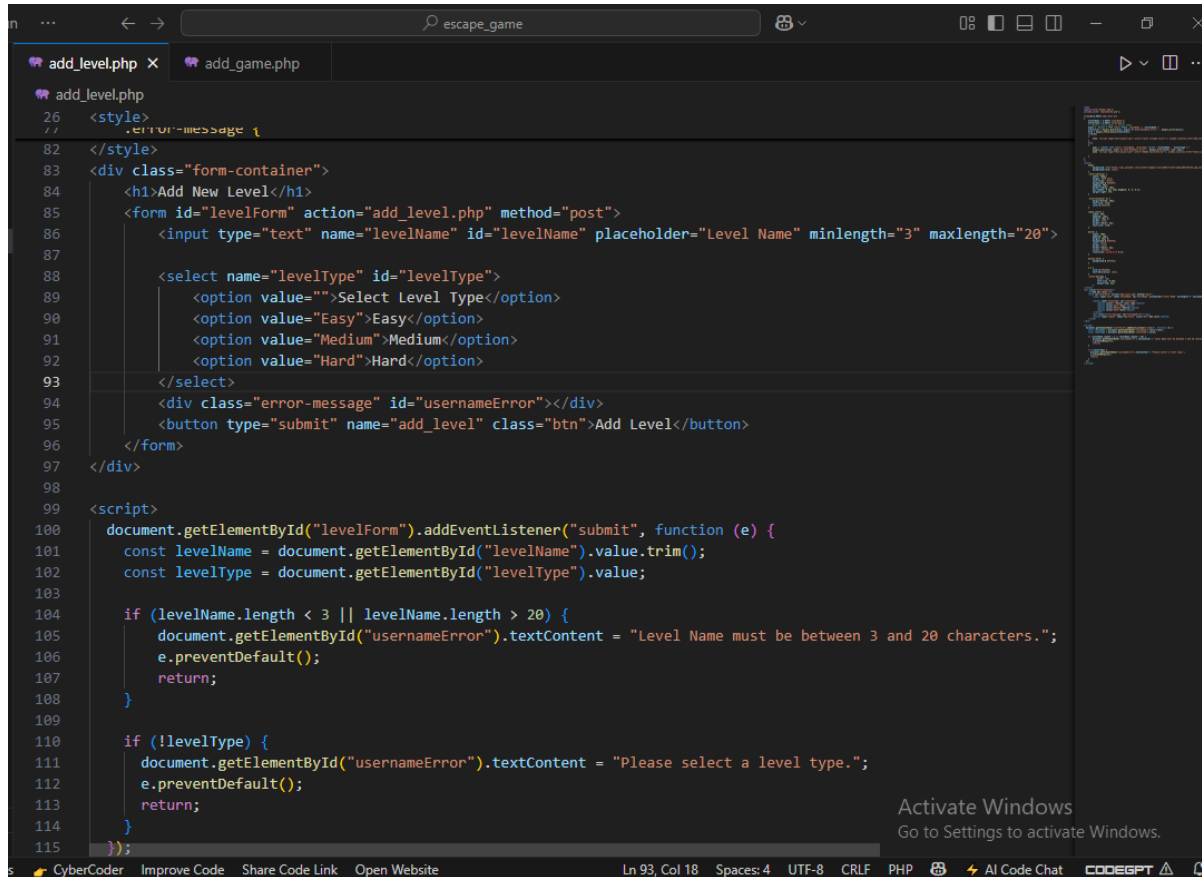


```
room3.php X
room3.php
3  <head>
9  <style>
94  {
104  font-size: 16px;
105  transition: background 0.3s;
106  }
107  </style>
108 </head>
109 <body>
110 <!-- نافذة الشروط -->
111 <div id="level-up-popup" class="popup">
112   <div class="popup-content">
113     <h2>Level Up Requirements</h2>
114     <p>You need at least 25 points and must finish the questions within the time limit!</p>
115     <button id="continue-btn">Next</button>
116     <button id="retry-btn">Retry</button>
117   </div>
118 </div>
119 <button id="pause-btn">⏸ Pause</button>
120 <button id="endButton">🚪 Exit</button>
121 <div id="timer">Time: 60</div>
122 <div id="score">Score: 0</div>
123 <div id="questionBox">
124   <p id="questionText"></p>
125   <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
126   <!-- <button onclick="answerQuestion(true)">Correct</button>
127   <button onclick="answerQuestion(false)">Wrong</button> -->
128 </div>
129
130 <div id="retryExitBox">
131   <p>Your score is less than 25! Would you like to try again or exit?</p>
132   <button onclick="tryAgain()">Try Again</button>
133   <button onclick="exitGame()">Exit</button>
134 </div>
135
136 <script>...
496 </script>
497 </body>
```

Figure 4. 24: Room 3 Code

4.6.8 Add Level for Admin Code

As shown in Figure 4.25, this Code lets admins create new game levels by entering the name, difficulty, and description. Built with HTML, PHP, and JavaScript, it stores data in MySQL after validation.



```
26 <style>
27 // .error-message {
28
29 </style>
30
31 <div class="form-container">
32   <h1>Add New Level</h1>
33   <form id="levelForm" action="add_level.php" method="post">
34     <input type="text" name="levelName" id="levelName" placeholder="Level Name" minlength="3" maxlength="20">
35
36     <select name="levelType" id="levelType">
37       <option value="">Select Level Type</option>
38       <option value="Easy">Easy</option>
39       <option value="Medium">Medium</option>
40       <option value="Hard">Hard</option>
41     </select>
42
43     <div class="error-message" id="usernameError"></div>
44     <button type="submit" name="add_level" class="btn">Add Level</button>
45   </form>
46 </div>
47
48 <script>
49
50 document.getElementById("levelForm").addEventListener("submit", function (e) {
51   const levelName = document.getElementById("levelName").value.trim();
52   const levelType = document.getElementById("levelType").value;
53
54   if (levelName.length < 3 || levelName.length > 20) {
55     document.getElementById("usernameError").textContent = "Level Name must be between 3 and 20 characters.";
56     e.preventDefault();
57     return;
58   }
59
60   if (!levelType) {
61     document.getElementById("usernameError").textContent = "Please select a level type.";
62     e.preventDefault();
63     return;
64   }
65
66 });
```

Figure 4. 25: Add Level for Admin Code

4.7.9 Admin Add Puzzle Code

As shown in Figure 4.26, this Code allows admins to add puzzles with questions, hints, and answers. JavaScript handles validation, while PHP and MySQL manage data storage.

```
h ... ← → escape_game
add_level.php add_game.php X
add_game.php
30 <style>
81 .error-message {
84     margin-top: 5px;
85 }
86 </style>
87 <div class="form-container">
88   <h1>Add New Game</h1>
89   <form id="gameForm" action="add_game.php" method="post">
90     <input type="text" name="gameName" id="gameName" placeholder="Game Name" minlength="3" maxlength="20">
91
92     <input type="number" name="endTime" id="endTime" placeholder="End Time">
93
94     <input type="number" name="score" id="score" placeholder="Score">
95
96     <select name="gameLevel" id="gameLevel">
97       <option value="">Select Game Level</option>
98       <?php
99         // make while loop to display levels
100         while ($level = $levels->fetch_assoc())
101         {
102           <?
103           <option value="<?= $level['id'] ?>"><?= $level['levelName'] ?></option>
104           <?php
105         }
106       <?
107     </select>
108     <div class="error-message" id="usernameError"></div>
109     <button type="submit" name="add_game" class="btn">Add Game</button>
110   </form>
111 </div>
112
113 <script>
114 document.getElementById("gameForm").addEventListener("submit", function (e) {
115   const gameName = document.getElementById("gameName").value.trim();
116   const endTime = document.getElementById("endTime").value.trim();
117   const score = document.getElementById("score").value.trim();
118   const gameLevel = document.getElementById("gameLevel").value;
119
```

Figure 4. 26: Admin Add Puzzle Code

Chapter 5: System Testing

5.1 Introduction

Testing and evaluation are the process by which a system or components are compared to requirements and specifications through testing. The results are evaluated to assess design progress, performance, supportability, etc. In this section, we will go through some of our platform's testing and evaluation stages.

5.2 Functional Testing

Functional testing is the method quality assurance teams use to verify whether a software application operates according to predefined requirements. Its focus is solely on determining if the system functions correctly. In the following section, we will explore the different types of functional testing.

5.2.1 Unit Testing

Unit testing is conducted by developers who write scripts to test whether individual components or modules of the software meet the specified requirements. This involves writing code that calls the methods within each module and verifying that the returned values align with the expected results.

5.2.2 Integration Testing

Integration testing is conducted when a system consists of multiple functional modules that need to work together. Its purpose is to verify that these individual modules interact correctly and perform as expected when integrated, ensuring smooth collaboration between them.

5.2.3 Regression Testing

Regression testing is conducted to verify that changes made to the code, such as adding new features or fixing bugs, have not negatively impacted existing functionality. It ensures that the software continues to work as expected after modifications, preventing unintended disruptions in its operation.

5.3 System Testing

System testing provides an overview of the evaluated system features, along with a detailed table listing all test cases. This table includes the objectives, inputs, expected results, pass or fail criteria, and the actual outcomes of each test.

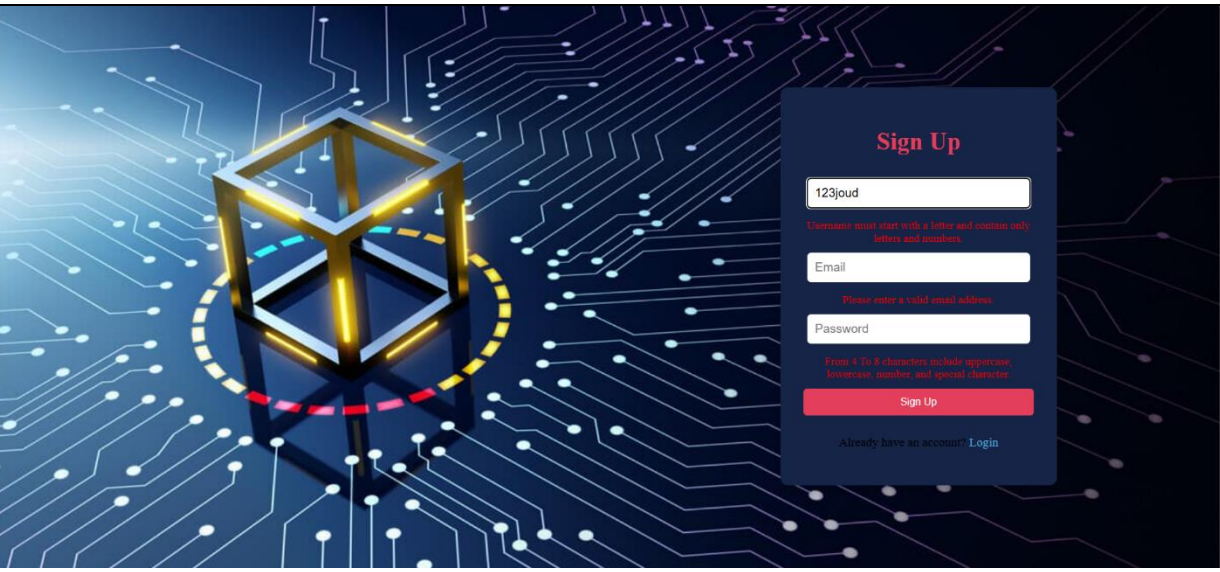
5.3.1 Test Cases

In this section, we will examine four test cases. The first test case involves an invalid sign-up attempt, where the username field is left empty. The second test case involves an invalid sign-up attempt, where the password field is left empty. The third test case concerns an invalid login, where the email field is left empty. The fourth test case concerns an invalid login, where the password field is left empty. The fifth test case concerns an email error, where the email is not correct. The sixth test case concerns password error, where the password is not correct. The seventh test case concerns an invalid add puzzle, where the game name field is left empty. The eighth test case concerns an invalid add level, where the level name field is left empty.

1. **Test case:** an invalid sign-up attempt, where the username field is left empty. As shown in Table 5.1.

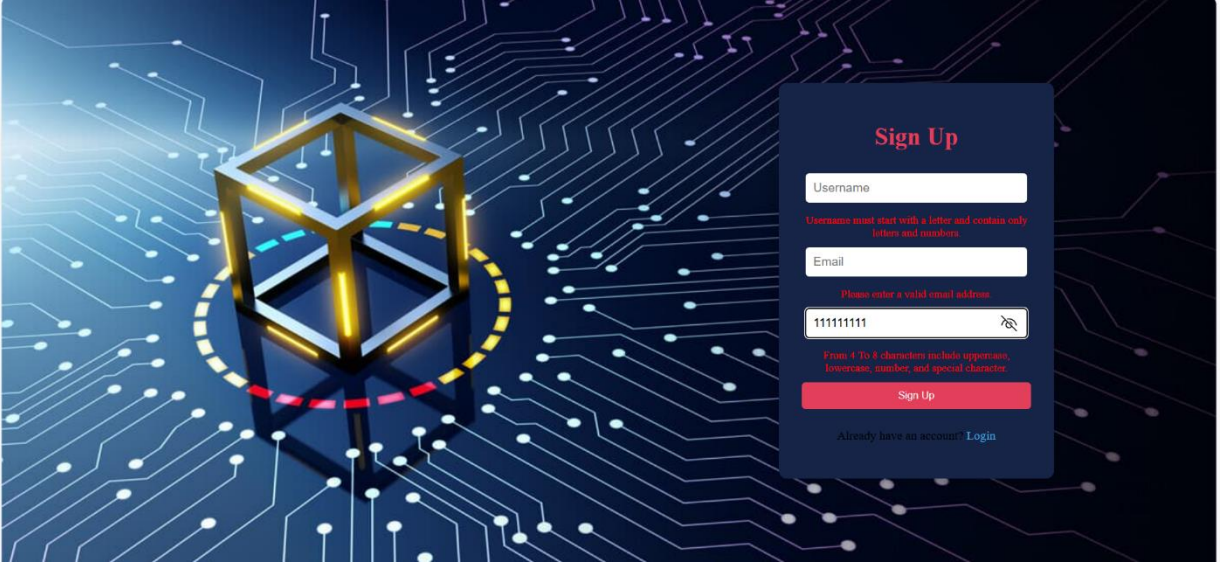
Table 5. 1: Test case: an invalid sign-up attempt, where the username field is left empty

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid Sign-up	To prevent him from accessing the login page.	Empty username	Show notification "Please fill out this field"	Pass

Test Result	
-------------	--

2. **Test case:** an invalid sign-up attempt, where the password field is left empty. As shown in Table 5.2.

Table 5. 2: Test case: an invalid sign-up attempt, where the password field is left empty

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid Sign-up	To prevent him from accessing the login page.	Empty password	Show notification “Please fill out this field”	Pass
Test Result				

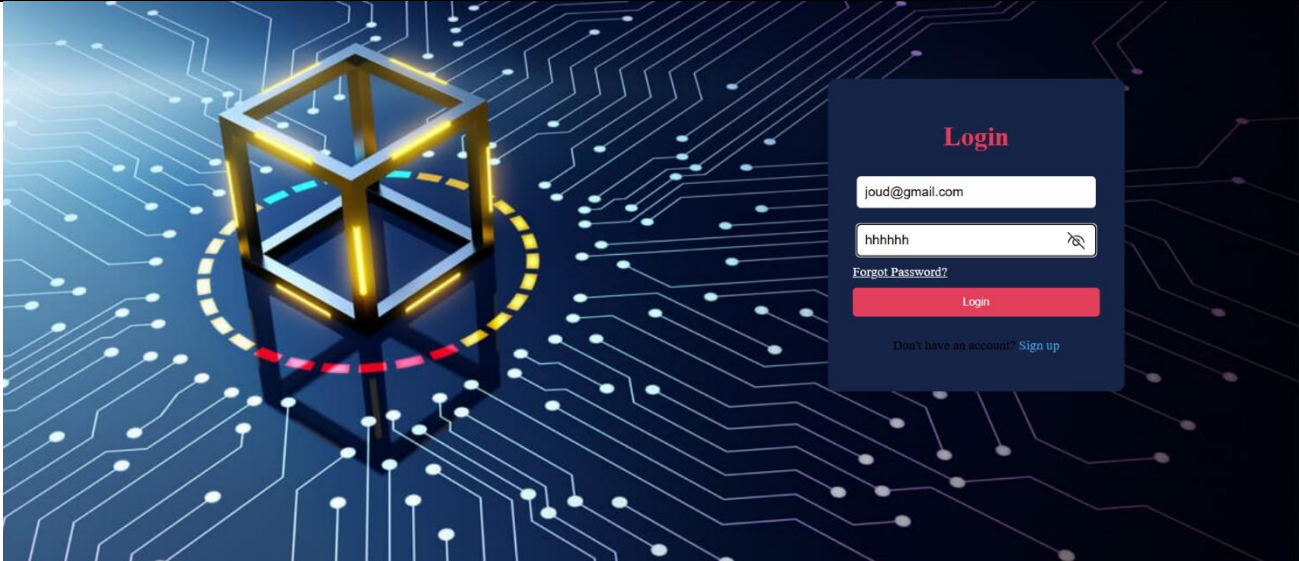
3. **Test Case:** an invalid login, where the email field is left empty. As shown in Table 5.3.

Table 5. 3: Test Case: an invalid login, where the email field is left empty

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid Login	To prevent him from accessing the home page.	Empty email	Show notification “Please fill out this field”	Pass
Test Result				

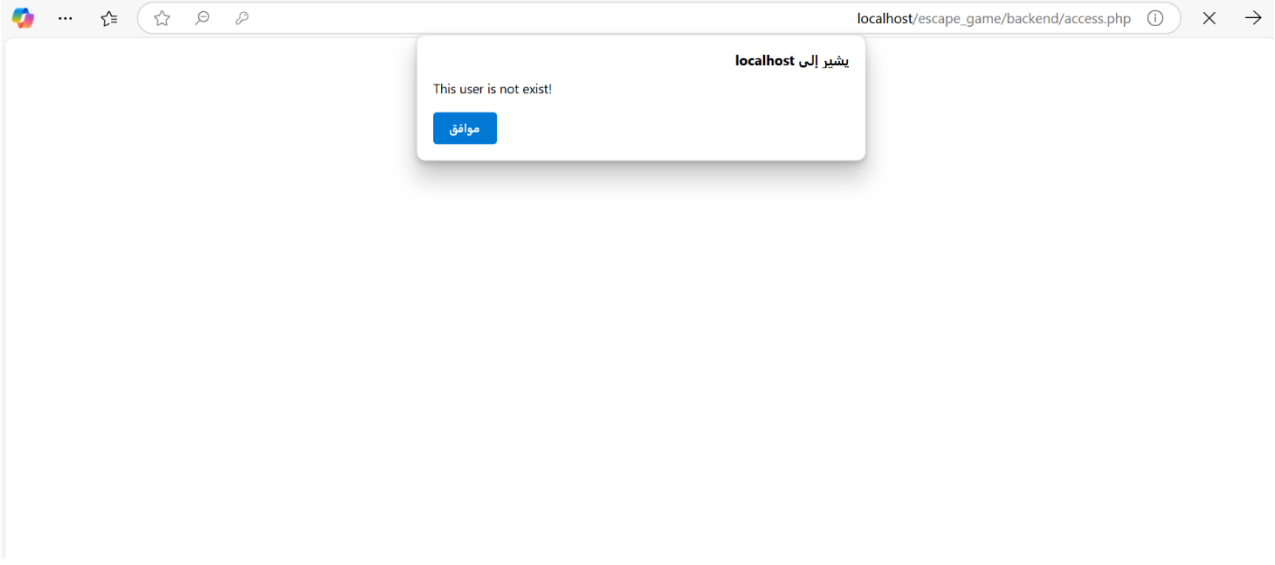
4. **Test Case:** an invalid login, where the password field is left empty. As shown in Table 5.4.

Table 5. 4: Test Case: an invalid login, where the password field is left empty

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid Login	To prevent him from accessing the home page.	Empty password	Show notification “Please fill out this field”	Pass
Test Result				

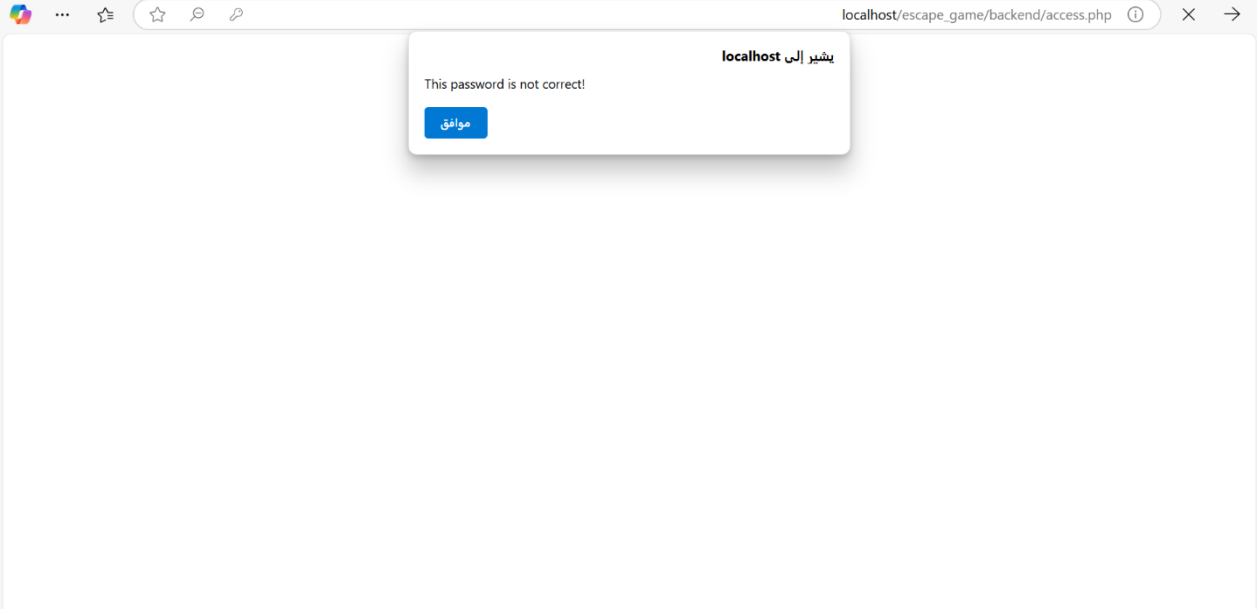
5. **Test Case:** email error, where the email is not correct. As shown in Table 5.5.

Table 5. 5: Test Case: email error, where the email is not correct

Test case	Purpose	Input	Expected output	Pass/Fail
Email error	To prevent him from accessing the home page.	Incorrect email	Show notification “Please put the correct email.”	Pass
Test Result				

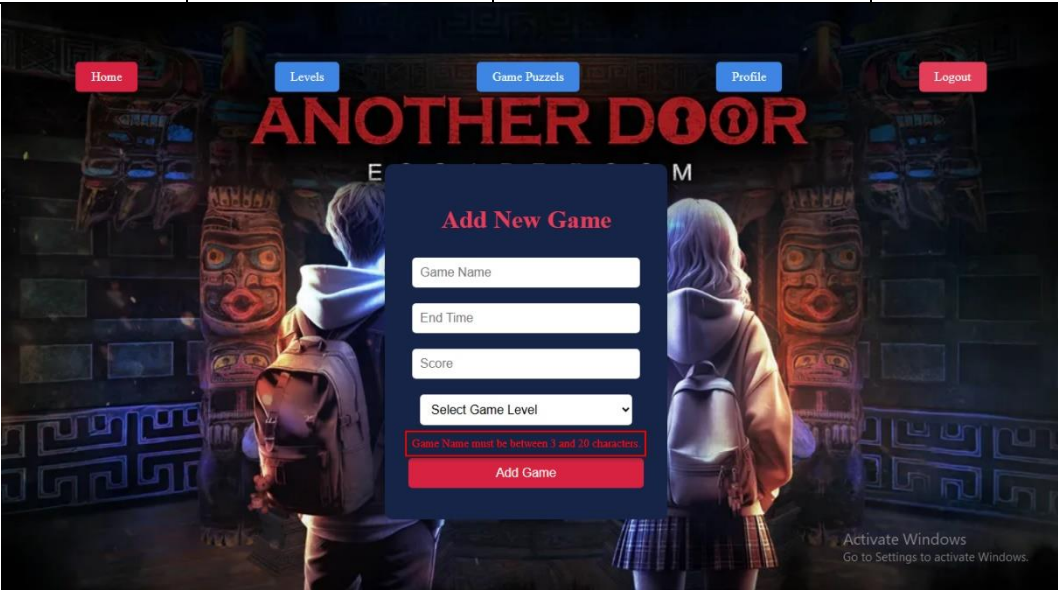
6. **Test Case:** password error, where the password is not correct. As shown in Table 5.6.

Table 5. 6: Test Case: password error, where the password is not correct

Test case	Purpose	Input	Expected output	Pass/Fail
Password error	To prevent him from accessing the home page.	Incorrect Password	Show notification “Please put the correct Password.”	Pass
Test Result				

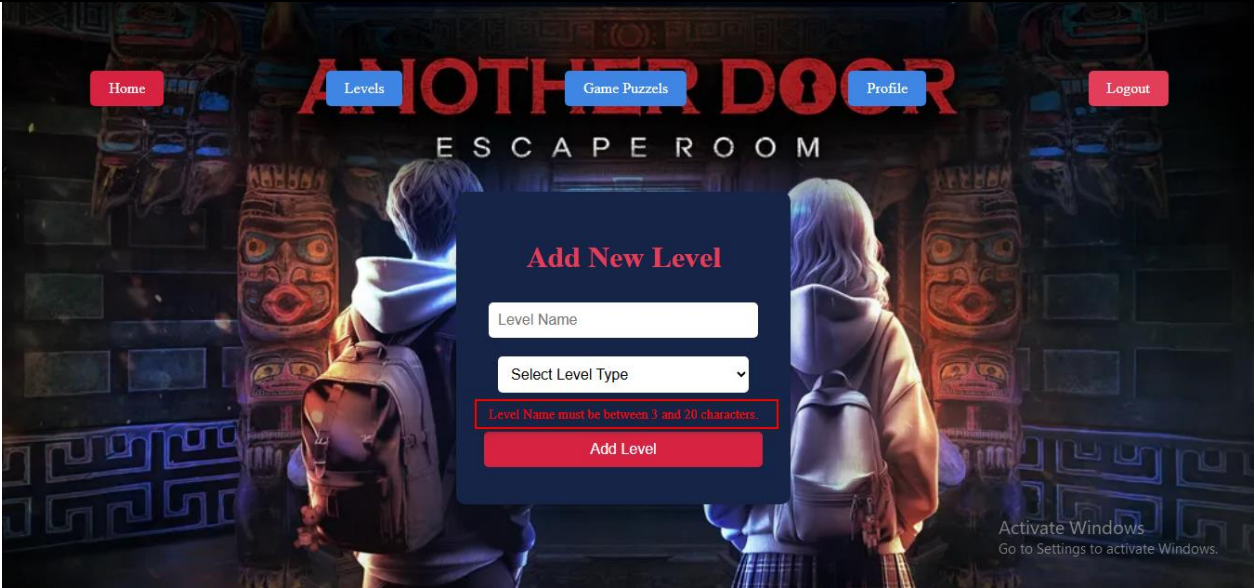
7. **Test Case:** an invalid add puzzle, where the game name field is left empty. As shown in Table 5.7.

Table 5. 7: Test Case: an invalid add puzzle

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid add puzzle	To prevent him from adding a puzzle	Empty game name field	Show notification “Please fill out this field”	Pass
Test Result				

8. **Test Case:** an invalid add level, where the level name field is left empty. As shown in Table 5.8.

Table 5. 8: Test Case: an invalid add level

Test case	Purpose	Input	Expected output	Pass/Fail
Invalid add level	To prevent him from adding a new level	Empty level name field	Show notification “Please fill out this field”	Pass
Test Result				

Chapter 6: Conclusion and Future Works

6.1 Conclusion

This project successfully develops an interactive cybersecurity-themed escape room game to enhance awareness among university students. By integrating React, JavaScript, PHP, and MySQL, the platform offers an engaging and educational experience with puzzles of varying difficulty levels. The project highlights the importance of cybersecurity education in an interactive format, ensuring players gain practical knowledge while enjoying the game.

6.1 Future Works

The future works of the project may include:

- **Expanding Puzzle Variety:** Introduce more complex and diverse cybersecurity challenges, including real-world threat simulations.
- **Multiplayer Mode:** Enable cooperative gameplay to enhance team-based learning and problem-solving skills.
- **Mobile Application Development:** Extend the platform to iOS and Android for greater accessibility.
- **AI-Driven Hints System:** Implement an AI-based assistant to provide personalized hints based on players' progress.

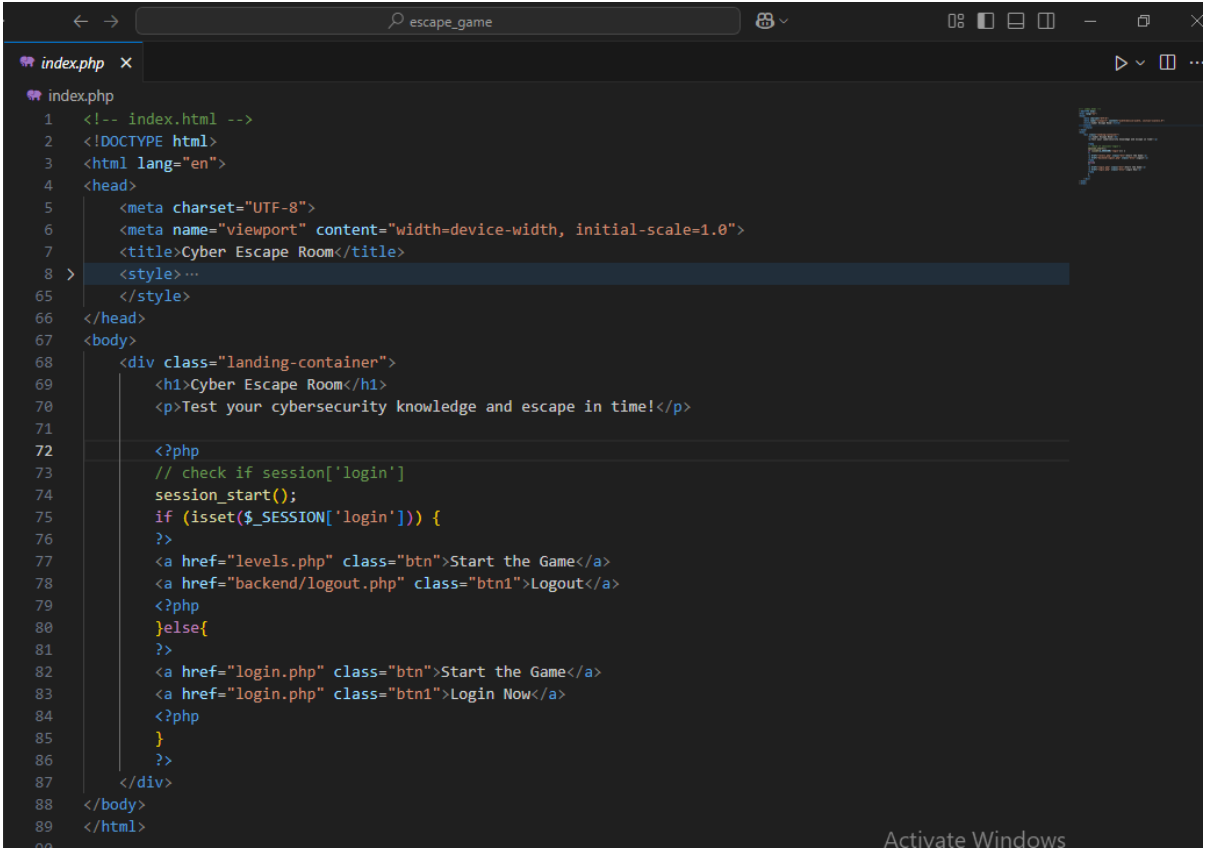
References

- [1] E. E. A. C. Markus Wiemker, "Escape Room Games," November 2015.
- [2] J. Buchner, M. Rüter and M. Kerres, "Learning with a digital escape room game: before or after instruction," *springer link*, 15 march 2022.
- [3] O. Heikkinen and J. Shumeyko, "Designing an escape room with the Experience Pyramid model," *Julkaisun pysyvä osoite on* , 2016.
- [4] S. Ian, Software Engineering, 11th ed., Addison-Wesley,, 2015.
- [5] E. Rooms, Escape Rooms, 2011.
- [6] "The Psychology of Escape Rooms: How They Engage Players," *Journal of Gaming & Virtual Worlds*, 2020.
- [7] temy.o, "Designing Puzzles for Escape Rooms," *Game Design Insights*, 2019.
- [8] "www.escaperoomdirectory.com," 2018. [Online]. "Escape Rooms: The Ultimate Guide to Creating Your Own" by T. R. K. Smith
- [9] This book provides insights into the design and creation of escape rooms, including themes, puzzles, and operational considerations.. "The Art of Game Design: A Book of Lenses" by Jesse Schell
- [10] A foundational text on game design principles that can be applied to creating engaging escape room experiences.
- [11] "The Lean Startup" by Eric Ries
- A guide on how to build a sustainable business model, which can be beneficial for the financial viability of your escape room.

Appendix

A. Sample Code

Home Code



```
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>Cyber Escape Room</title>
8     <style>...
65 </style>
66 </head>
67 <body>
68     <div class="landing-container">
69         <h1>Cyber Escape Room</h1>
70         <p>Test your cybersecurity knowledge and escape in time!</p>
71
72     <?php
73         // check if session['login']
74         session_start();
75         if (isset($_SESSION['login'])) {
76             ?>
77             <a href="levels.php" class="btn">Start the Game</a>
78             <a href="backend/logout.php" class="btn1">Logout</a>
79             <?php
80         }else{
81             ?>
82             <a href="login.php" class="btn">Start the Game</a>
83             <a href="login.php" class="btn1">Login Now</a>
84             <?php
85         }
86     ?>
87 </div>
88 </body>
89 </html>
```

Figure 4. 27: Home Code

Sign-up Code

```
escape_game

signup.php X
signup.php
1  <!-- signup.html -->
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Sign Up</title>
8  >
9      <style>...
10 </style>
11 </head>
12 <body>
13     <div class="form-container">
14         <h1>Sign Up</h1>
15         <form action="backend/access.php" method="post">
16             <input type="text" name="username" placeholder="Username" required>
17             <input type="email" name="email" placeholder="Email" required>
18             <input type="password" name="password" placeholder="Password" required>
19             <button type="submit" name="signup" class="btn">Sign Up</button>
20             <p>Already have an account? <a href="login.php">Login</a></p>
21         </form>
22     </div>
23 </body>
24 </html>
25
```

Figure 4. 28: Sign-up Code

Login Code

```
escape_game

login.php X
login.php
1  <!-- login.html -->
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Login</title>
8  >
9      <style>...
10 </style>
11 </head>
12 <body>
13     <div class="form-container">
14         <h1>Login</h1>
15         <form action="backend/access.php" method="post">
16             <input type="text" name="email" placeholder="Email" required>
17             <input type="password" name="password" placeholder="Password" required>
18             <button type="submit" name="login" class="btn">Login</button>
19             <p>Don't have an account? <a href="signup.php">Sign up</a></p>
20         </form>
21     </div>
22 </body>
23 </html>
24
```

Figure 4. 29: Login Code

Levels Code

```
escape_game

levels.php x
levels.php
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Choose Your Level</title>
7   <style>...
115 </style>
116 </head>
117 <body>
118   <div class="level-container">
119     <!-- Easy Level -->
120     <div class="level-box easy" onclick="startLevel('room1')">
121       <h2>Easy</h2>
122       <div class="icon">😊</div>
123       <p>Perfect for beginners. Start simple and have fun!</p>
124       <div class="difficulty">Difficulty: Easy</div>
125     </div>
126
127     <!-- Medium Level -->
128     <div class="level-box medium" onclick="startLevel('room2')">
129       <h2>Medium</h2>
130       <div class="icon">😬</div>
131       <p>Step up your game with intermediate challenges.</p>
132       <div class="difficulty">Difficulty: Medium</div>
133     </div>
134
135     <!-- Hard Level -->
136     <div class="level-box hard" onclick="startLevel('room3')">
137       <h2>Hard</h2>
138       <div class="icon">🔥</div>
139       <p>Ready for a real challenge? Test your skills!</p>
140       <div class="difficulty">Difficulty: Hard</div>
141     </div>
142   </div>
143
144   <script>
145     function startLevel(level) {
146       // Redirect to the respective level's page
147       window.location.href = `${level}.php`;
148     }
149   </script>
150 </body>
151 </html>
152
```

Figure 4. 30: Levels Code

Room 1 Code

```
room1.php
<head>
</style>
</head>
<body>
<!-- نافذة الشروط -->
<div id="level-up-popup" class="popup">
  <div class="popup-content">
    <h2>Level Up Requirements</h2>
    <p>You need at least 25 points and must finish the questions within the time limit!</p>
    <button id="continue-btn">Next</button>
    <button id="retry-btn">Retry</button>
  </div>
</div>

<button id="pause-btn">|| Pause</button>
<button id="endButton"> Exit</button>

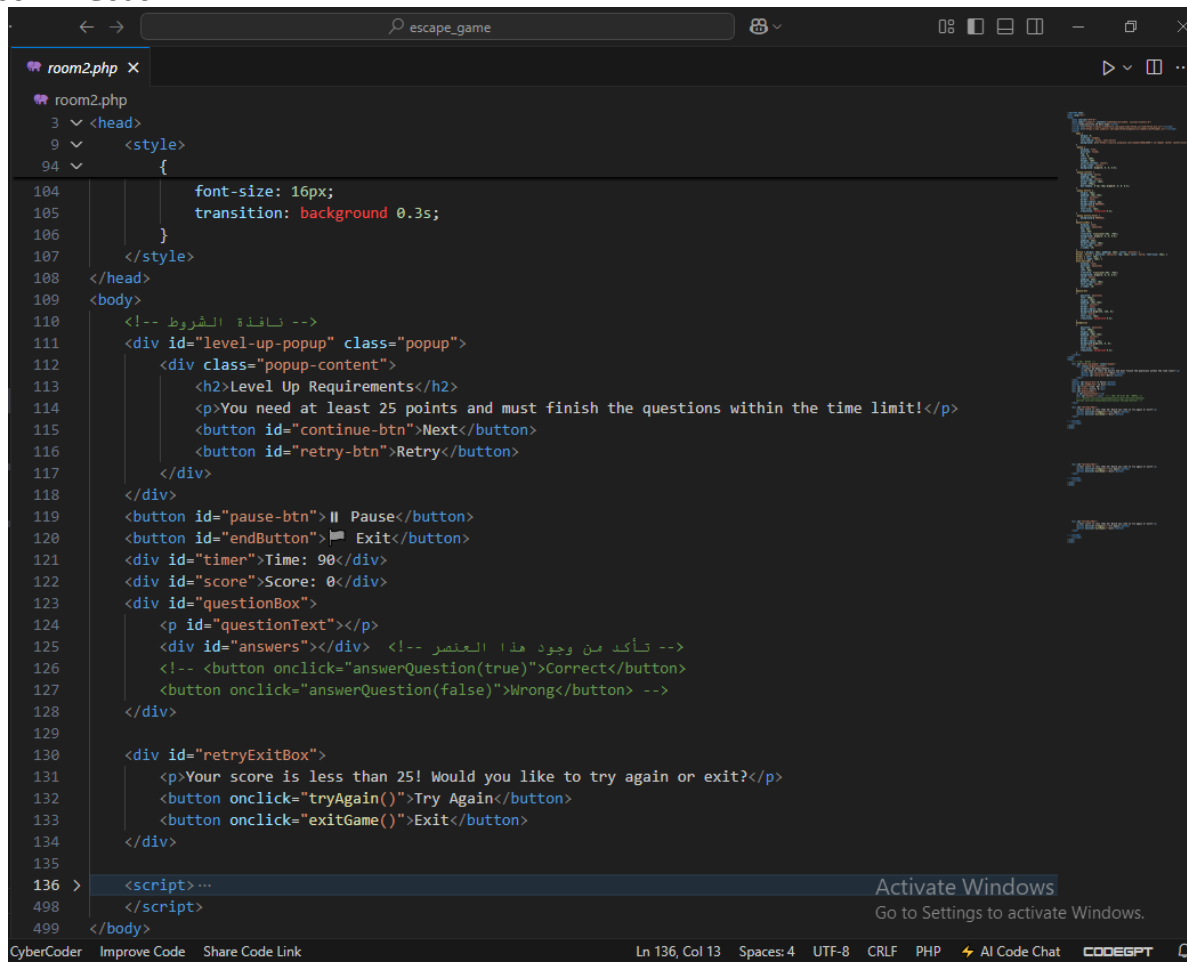
<div id="timer">Time: 120</div>
<div id="score">Score: 0</div>
<div id="questionBox">
  <p id="questionText"></p>
  <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
  <!-- <button onclick="answerQuestion(true)">Correct</button>
  <button onclick="answerQuestion(false)">Wrong</button> -->
</div>

<div id="retryExitBox">
  <p>Your score is less than 25! Would you like to try again or exit?</p>
  <button onclick="tryAgain()">Try Again</button>
  <button onclick="exitGame()">Exit</button>
</div>

<script>...
</script>
</body>
</html>
```

Figure 4. 31: Room 1 Code

Room 2 Code



```
room2.php X
room2.php
3 <head>
9 <style>
94 {
104     font-size: 16px;
105     transition: background 0.3s;
106 }
107 </style>
108 </head>
109 <body>
110 <!-- نافذة الشروط -->
111 <div id="level-up-popup" class="popup">
112     <div class="popup-content">
113         <h2>Level Up Requirements</h2>
114         <p>You need at least 25 points and must finish the questions within the time limit!</p>
115         <button id="continue-btn">Next</button>
116         <button id="retry-btn">Retry</button>
117     </div>
118 </div>
119 <button id="pause-btn">⏸ Pause</button>
120 <button id="endButton">🏠 Exit</button>
121 <div id="timer">Time: 90</div>
122 <div id="score">Score: 0</div>
123 <div id="questionBox">
124     <p id="questionText"></p>
125     <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
126     <!-- <button onclick="answerQuestion(true)">Correct</button>
127     <button onclick="answerQuestion(false)">Wrong</button> -->
128 </div>
129
130 <div id="retryExitBox">
131     <p>Your score is less than 25! Would you like to try again or exit?</p>
132     <button onclick="tryAgain()">Try Again</button>
133     <button onclick="exitGame()">Exit</button>
134 </div>
135
136 <script>...
498 </script>
499 </body>
```

Activate Windows
Go to Settings to activate Windows.

CyberCoder Improve Code Share Code Link Ln 136, Col 13 Spaces: 4 UTF-8 CRLF PHP AI Code Chat CODEGPT

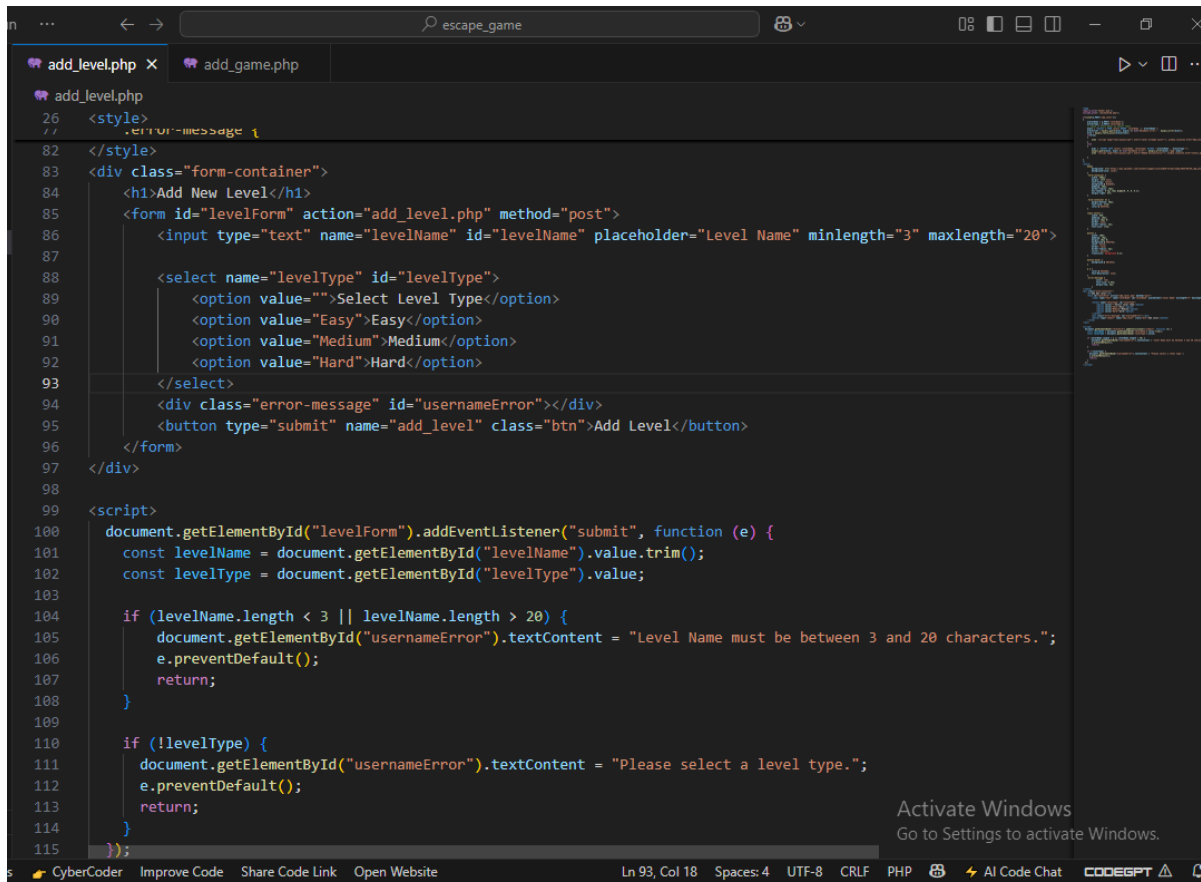
Figure 4. 32: Room 2 Code

Room 3 Code

```
room3.php
3 <head>
9 <style>
94 {
104 font-size: 16px;
105 transition: background 0.3s;
106 }
107 </style>
108 </head>
109 <body>
110 <!-- نافذة الشروط -->
111 <div id="level-up-popup" class="popup">
112 <div class="popup-content">
113 <h2>Level Up Requirements</h2>
114 <p>You need at least 25 points and must finish the questions within the time limit!</p>
115 <button id="continue-btn">Next</button>
116 <button id="retry-btn">Retry</button>
117 </div>
118 </div>
119 <button id="pause-btn">⏸ Pause</button>
120 <button id="endButton">🚪 Exit</button>
121 <div id="timer">Time: 60</div>
122 <div id="score">Score: 0</div>
123 <div id="questionBox">
124 <p id="questionText"></p>
125 <div id="answers"></div> <!-- تأكد من وجود هذا العنصر -->
126 <!-- <button onclick="answerQuestion(true)">Correct</button>
127 <button onclick="answerQuestion(false)">Wrong</button> -->
128 </div>
129
130 <div id="retryExitBox">
131 <p>Your score is less than 25! Would you like to try again or exit?</p>
132 <button onclick="tryAgain()">Try Again</button>
133 <button onclick="exitGame()">Exit</button>
134 </div>
135
136 <script>...
496 </script>
497 </body>
```

Figure 4. 33: Room 3 Code

Add Level for Admin Code



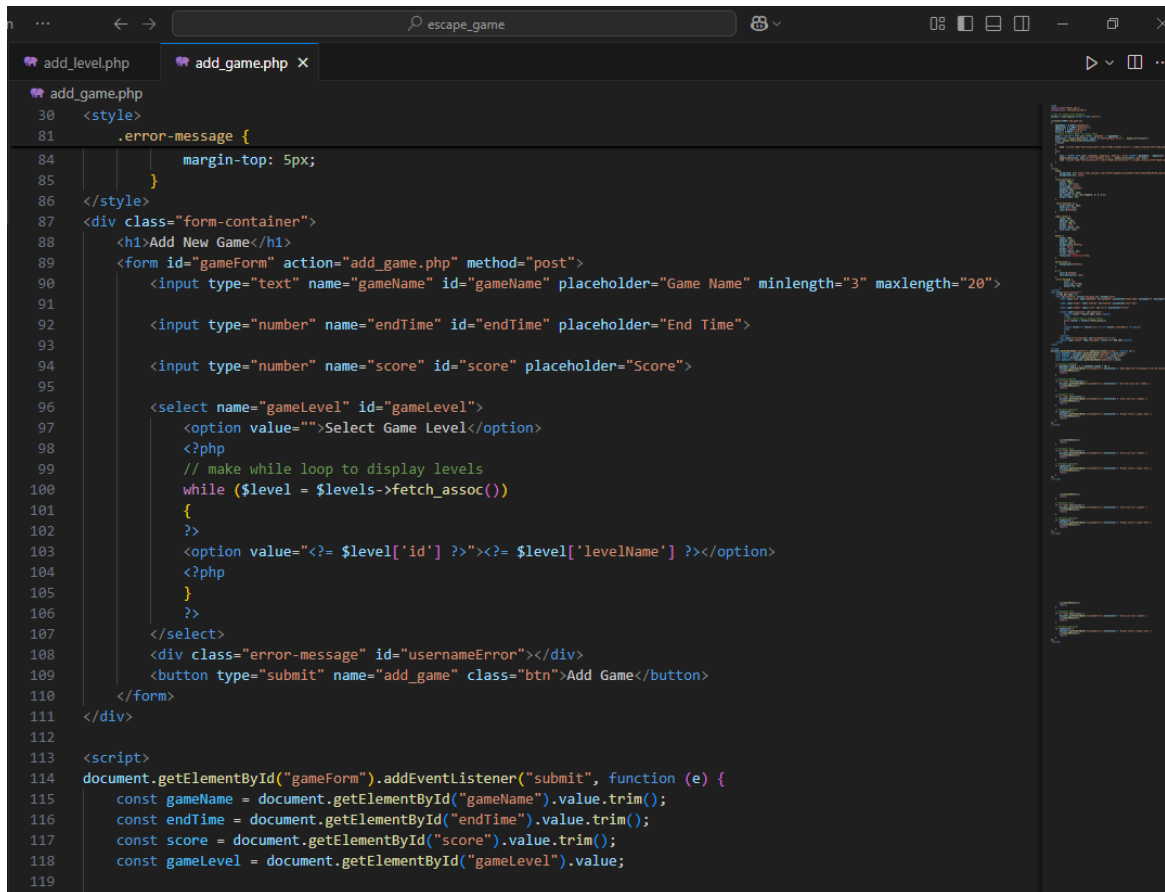
```
26 <style>
// .error-message {
27
28 </style>
29
30 <div class="form-container">
31 <h1>Add New Level</h1>
32 <form id="levelForm" action="add_level.php" method="post">
33 <input type="text" name="levelName" id="levelName" placeholder="Level Name" minlength="3" maxlength="20">
34
35 <select name="levelType" id="levelType">
36 <option value="">Select Level Type</option>
37 <option value="Easy">Easy</option>
38 <option value="Medium">Medium</option>
39 <option value="Hard">Hard</option>
40 </select>
41
42 <div class="error-message" id="usernameError"></div>
43 <button type="submit" name="add_level" class="btn">Add Level</button>
44 </form>
45 </div>
46
47 <script>
48 document.getElementById("levelForm").addEventListener("submit", function (e) {
49   const levelName = document.getElementById("levelName").value.trim();
50   const levelType = document.getElementById("levelType").value;
51
52   if (levelName.length < 3 || levelName.length > 20) {
53     document.getElementById("usernameError").textContent = "Level Name must be between 3 and 20 characters.";
54     e.preventDefault();
55     return;
56   }
57
58   if (!levelType) {
59     document.getElementById("usernameError").textContent = "Please select a level type.";
60     e.preventDefault();
61     return;
62   }
63 });
64
```

Activate Windows
Go to Settings to activate Windows.

CyberCoder Improve Code Share Code Link Open Website Ln 93, Col 18 Spaces: 4 UTF-8 CRLF PHP AI Code Chat CODEGPT

Figure 4. 34: Add Level for Admin Code

Admin Add Puzzle Code



```
add_level.php  add_game.php X
add_game.php
30 <style>
81 .error-message {
84     margin-top: 5px;
85 }
86 </style>
87 <div class="form-container">
88   <h1>Add New Game</h1>
89   <form id="gameForm" action="add_game.php" method="post">
90     <input type="text" name="gameName" id="gameName" placeholder="Game Name" minlength="3" maxlength="20">
91
92     <input type="number" name="endTime" id="endTime" placeholder="End Time">
93
94     <input type="number" name="score" id="score" placeholder="Score">
95
96     <select name="gameLevel" id="gameLevel">
97       <option value="">Select Game Level</option>
98       <?php
99         // make while loop to display levels
100         while ($level = $levels->fetch_assoc())
101         {
102           ?>
103           <option value="<?= $level['id'] ?>"><?= $level['levelName'] ?></option>
104           <?php
105           }
106         ?>
107       </select>
108     <div class="error-message" id="usernameError"></div>
109     <button type="submit" name="add_game" class="btn">Add Game</button>
110   </form>
111 </div>
112
113 <script>
114 document.getElementById("gameForm").addEventListener("submit", function (e) {
115   const gameName = document.getElementById("gameName").value.trim();
116   const endTime = document.getElementById("endTime").value.trim();
117   const score = document.getElementById("score").value.trim();
118   const gameLevel = document.getElementById("gameLevel").value;
119 }
```

Figure 4. 35: Admin Add Puzzle Code